

LLMs & CodeGen

Gabriel Synnaeve

`gabriel.synnaeve@gmail.com`

MVA, March 19, 2026

About LLMs

LLM 101

- $P(\text{next token} \mid \text{all previous tokens})$
 - For all tokens
 - For **lots of text**
- `<bos> the cat sat on the mat <eos>`
Teacher forcing:

`<bos> ???`

`<bos> the ???`

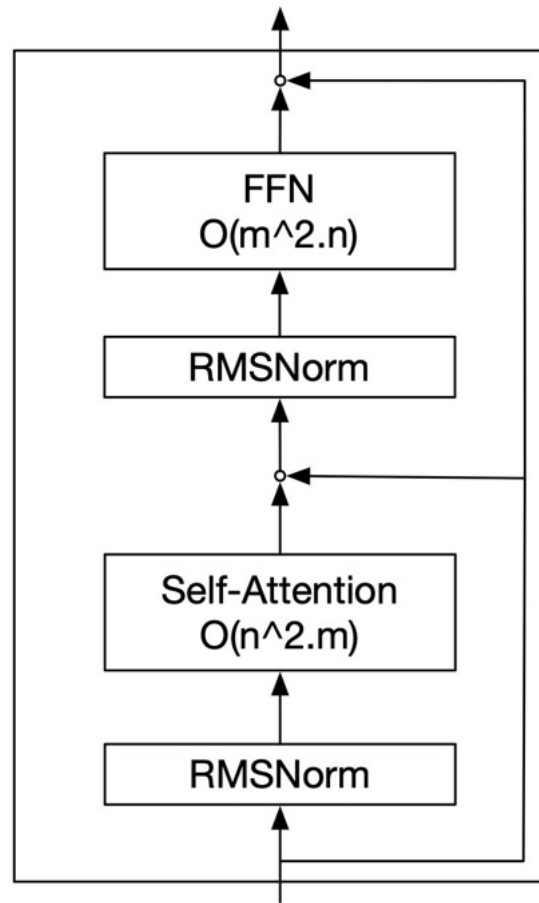
`<bos> the cat ???`

`<bos> the cat sat ???`

`<bos> the cat sat on ???`

`<bos> the cat sat on the ???`

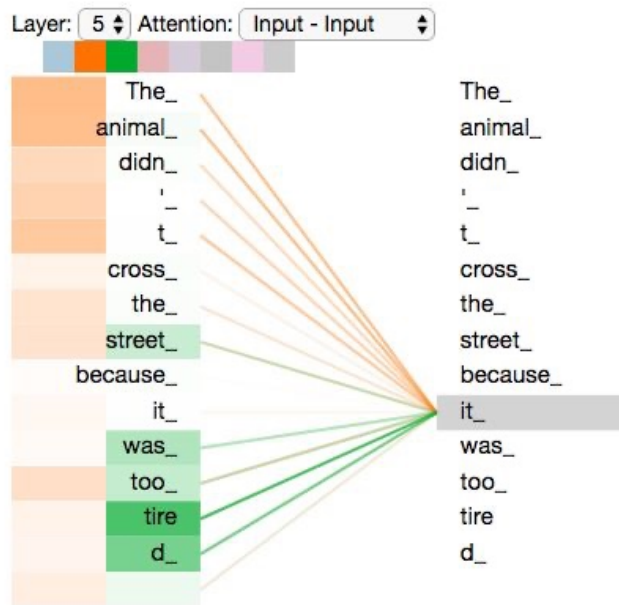
`<bos> the cat sat on the mat ???`



Transformers / Self-attention

$\text{softmax}(Q.K').V$

Multi-head -> do it for lots of vectors in parallel and then aggregate again



On Transformers vs. RNN

- RNN: $h(t) = f(U.x(t) + W.h(t-1))$
- The RNN can't know at token K which tokens between 1 and K will be important to predict token N with $N > K$, self-attention can.
- The RNN computation requires hiddens to be computed in sequence so the computation graph is a comb, while self-attention makes GPUs happy.
- Most state space models (SSM) are like & improve over RNN.

LLM 101 (illustrated on Llama 2)

GPT-3 Codex

This is a tokenizer

Clear

Show example

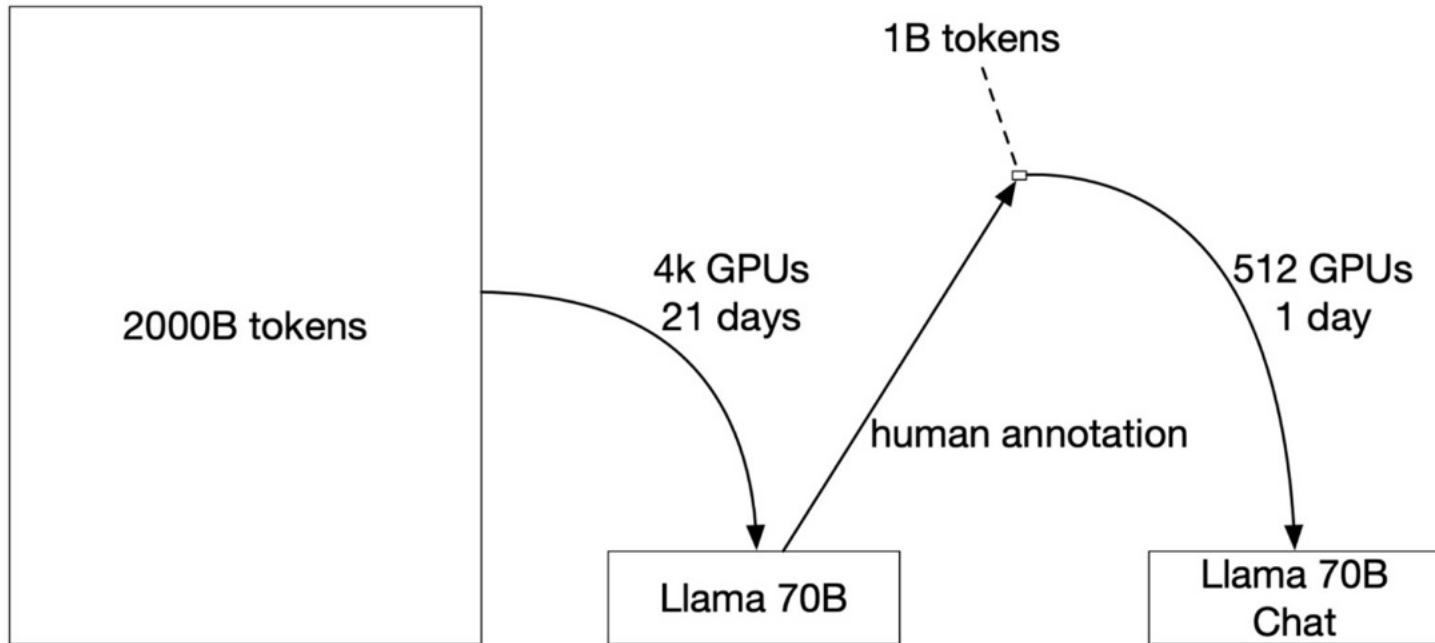
Tokens

Characters

5

19

This is a tokenizer



Overview of the modern cycle of life of an LLM

- Training

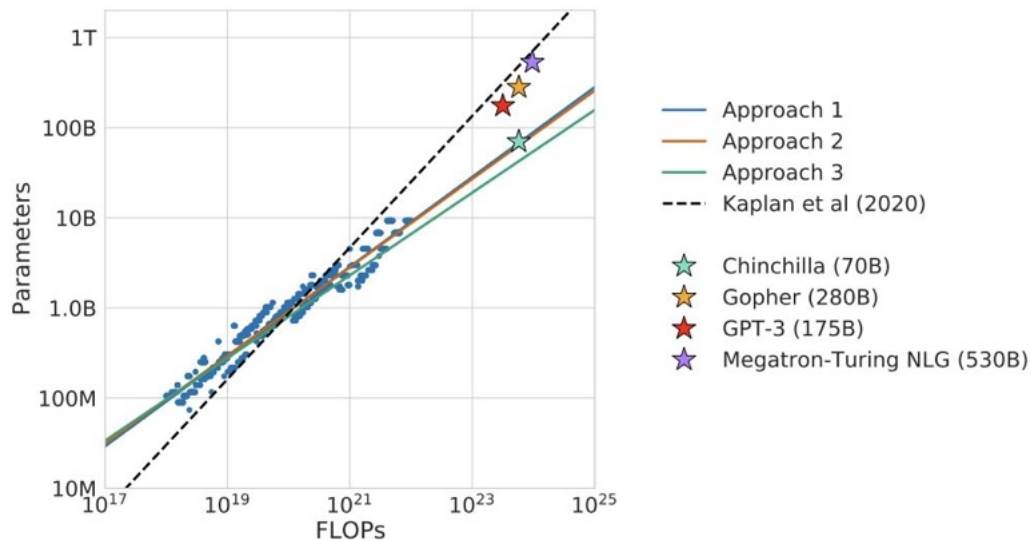
- Pretraining – learning syntax, facts, representations
- Midtraining – becoming an expert in specific topics (including long context training)
- Posttraining – learning skills, interfaces (tools, products), alignment with UX

- Inference

- Chat / assistant – you->AI->you->AI->\$\$\$
- Agent – you->AI->environment->AI->environment->AI->environment->\$\$\$
- Generating to train (synthetic data, RLHF, RLEF)

On scaling laws

- 15T tokens for Llama 3 8B $\rightarrow$ 200B tokens are “Chinchilla optimal”, but **this only considers training optimality!**
- Put it into context: before Chinchilla we were training 500B params models on 700B tokens.
- #tokens $\neq$ #unique tokens $\neq$ #updates



Llama 3.1 <https://ai.meta.com/blog/meta-llama-3-1/>

Category	Benchmark	Llama 3 8B	Gemma 2 9B	Mistral 7B	Llama 3 70B	Mixtral 8x22B	GPT 3.5 Turbo	Llama 3 405B	Nemotron 4 340B	GPT-4 ⁽⁰¹²⁵⁾	GPT-4o	Claude 3.5 Sonnet
General	MMLU ^(5-shot)	69.4	72.3	61.1	83.6	76.9	70.7	87.3	82.6	85.1	89.1	89.9
	MMLU ^(0-shot, CoT)	73.0	72.3 [△]	60.5	86.0	79.9	69.8	88.6	78.7 [△]	85.4	88.7	88.3
	MMLU-Pro ^(5-shot, CoT)	48.3	—	36.9	66.4	56.3	49.2	73.3	62.7	64.8	74.0	77.0
	IFEval	80.4	73.6	57.6	87.5	72.7	69.9	88.6	85.1	84.3	85.6	88.0
Code	HumanEval ^(0-shot)	72.6	54.3	40.2	80.5	75.6	68.0	89.0	73.2	86.6	90.2	92.0
	MBPP EvalPlus ^(0-shot)	72.8	71.7	49.5	86.0	78.6	82.0	88.6	72.8	83.6	87.8	90.5
Math	GSM8K ^(8-shot, CoT)	84.5	76.7	53.2	95.1	88.2	81.6	96.8	92.3 [◇]	94.2	96.1	96.4 [◇]
	MATH ^(0-shot, CoT)	51.9	44.3	13.0	68.0	54.1	43.1	73.8	41.1	64.5	76.6	71.1
Reasoning	ARC Challenge ^(0-shot)	83.4	87.6	74.2	94.8	88.7	83.7	96.9	94.6	96.4	96.7	96.7
	GPQA ^(0-shot, CoT)	32.8	—	28.8	46.7	33.3	30.8	51.1	—	41.4	53.6	59.4
Tool use	BFCL	76.1	—	60.4	84.8	—	85.9	88.5	86.5	88.3	80.5	90.2
	Nexus	38.5	30.0	24.7	56.7	48.5	37.2	58.7	—	50.3	56.1	45.7
Long context	ZeroSCROLLS/QuALITY	81.0	—	—	90.5	—	—	95.2	—	95.2	90.5	90.5
	InfiniteBench/En.MC	65.1	—	—	78.2	—	—	83.4	—	72.1	82.5	—
	NIH/Multi-needle	98.8	—	—	97.5	—	—	98.1	—	100.0	100.0	90.8
Multilingual	MGSM ^(0-shot, CoT)	68.9	53.2	29.9	86.9	71.1	51.4	91.6	—	85.9	90.5	91.6

Table 2 Performance of finetuned Llama 3 models on key benchmark evaluations. The table compares the performance of the 8B, 70B, and 405B versions of Llama 3 with that of competing models. We **boldface** the best-performing model in each of three model-size equivalence classes. [△]Results obtained using 5-shot prompting (no CoT). [△]Results obtained without CoT. [◇]Results obtained using zero-shot prompting.

Llama 3.1 scaling laws

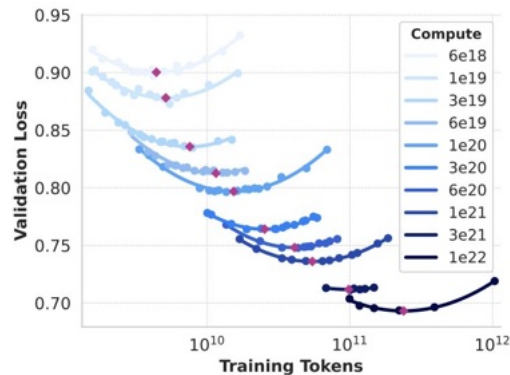


Figure 2 Scaling law IsoFLOPs curves between 6×10^{18} and 10^{22} FLOPs. The loss is the negative log-likelihood on a held-out validation set. We approximate measurements at each compute scale using a second degree polynomial.

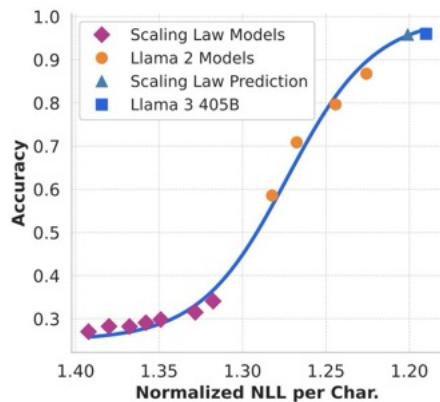
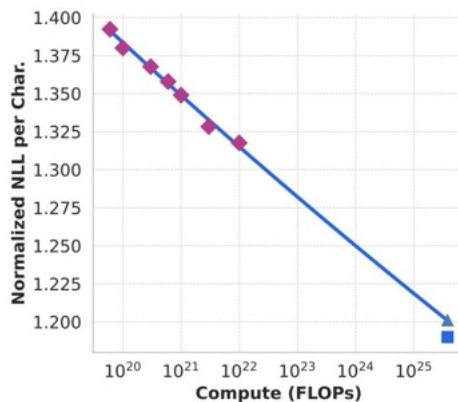
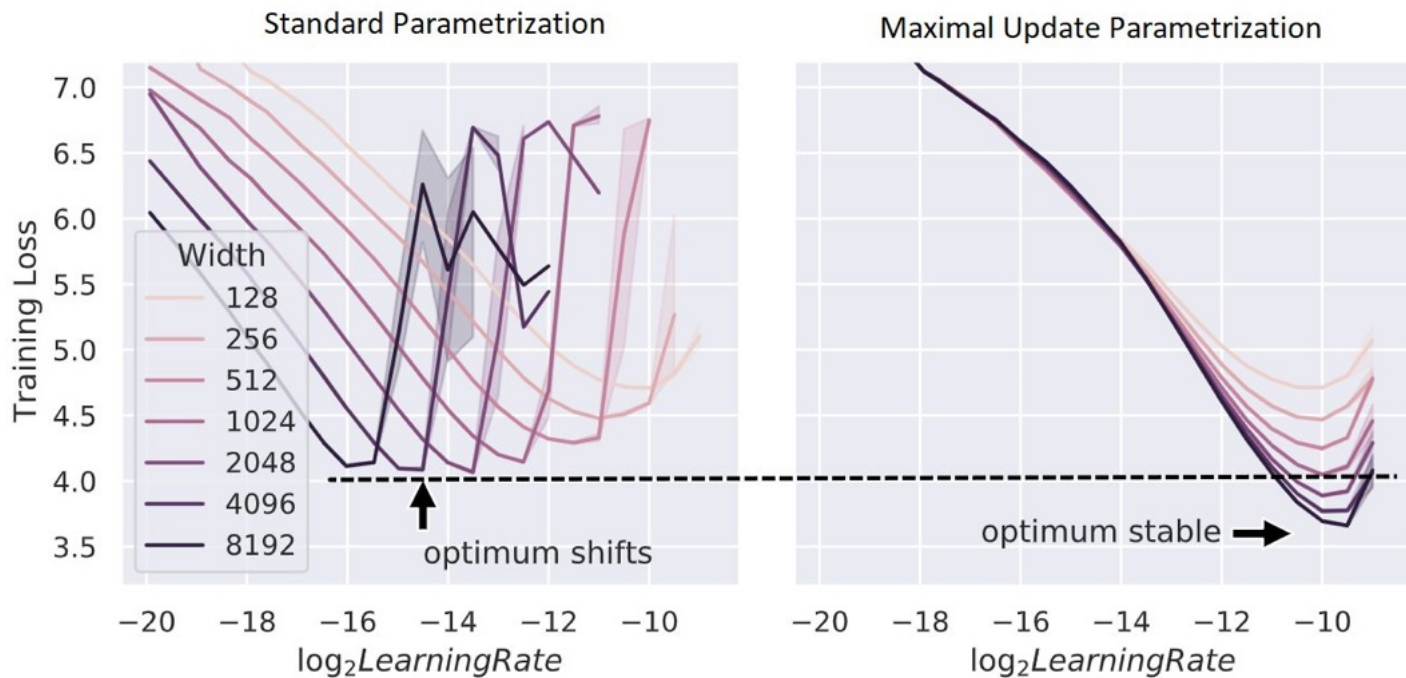


Figure 4 Scaling law forecast for ARC Challenge. *Left:* Normalized negative log-likelihood of the correct answer on the ARC Challenge benchmark as a function of pre-training FLOPs. *Right:* ARC Challenge benchmark accuracy as a function of the normalized negative log-likelihood of the correct answer. This analysis enables us to predict model performance on the ARC Challenge benchmark before pre-training commences. See text for details.

Maximal Update Parametrization (github.com/microsoft/mup)



	Input weights & all biases		Output weights		Hidden weights	
Init. Var.		$1/\text{fan_in}$	$1/\text{fan_in}^2$	$(1/\text{fan_in})$	$1/\text{fan_in}$	
SGD LR	fan_out	(1)	$1/\text{fan_in}$	(1)	1	
Adam LR		1	$1/\text{fan_in}$	(1)	$1/\text{fan_in}$	(1)

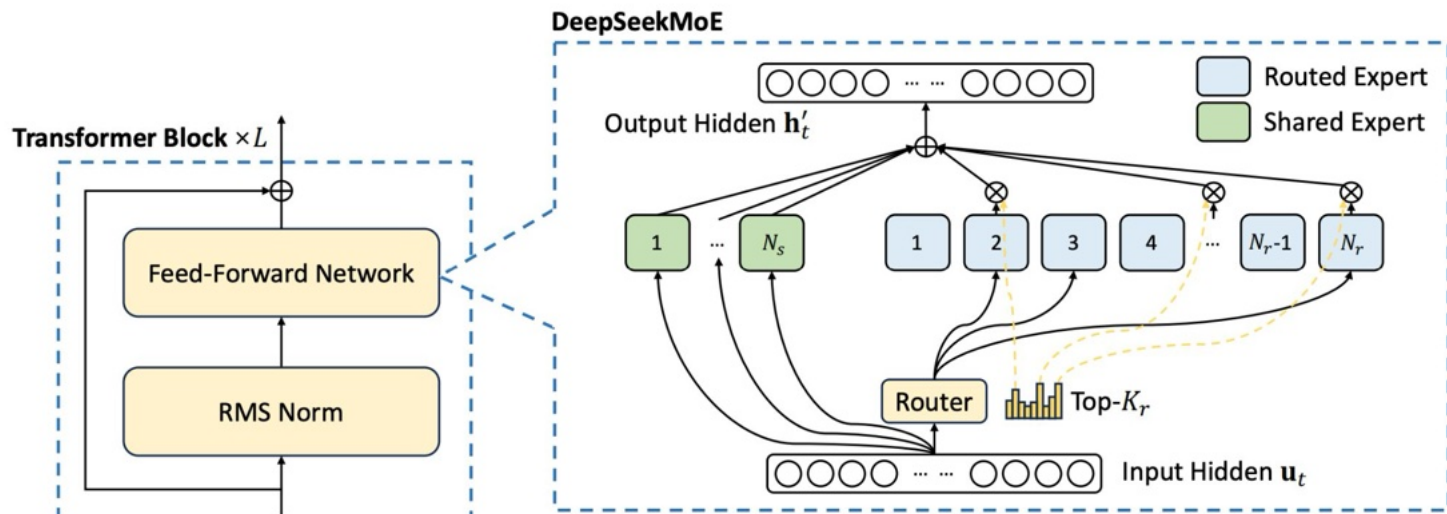
Deciphering a model definition (e.g. Llama 3)

	8B	70B	405B
Layers	32	80	126
Model Dimension	4,096	8192	16,384
FFN Dimension	14,336	28,672	53,248
Attention Heads	32	64	128
Key/Value Heads	8	8	8
Peak Learning Rate	3×10^{-4}	1.5×10^{-4}	8×10^{-5}
Activation Function	SwiGLU		
Vocabulary Size	128,000		
Positional Embeddings	RoPE ($\theta = 500,000$)		

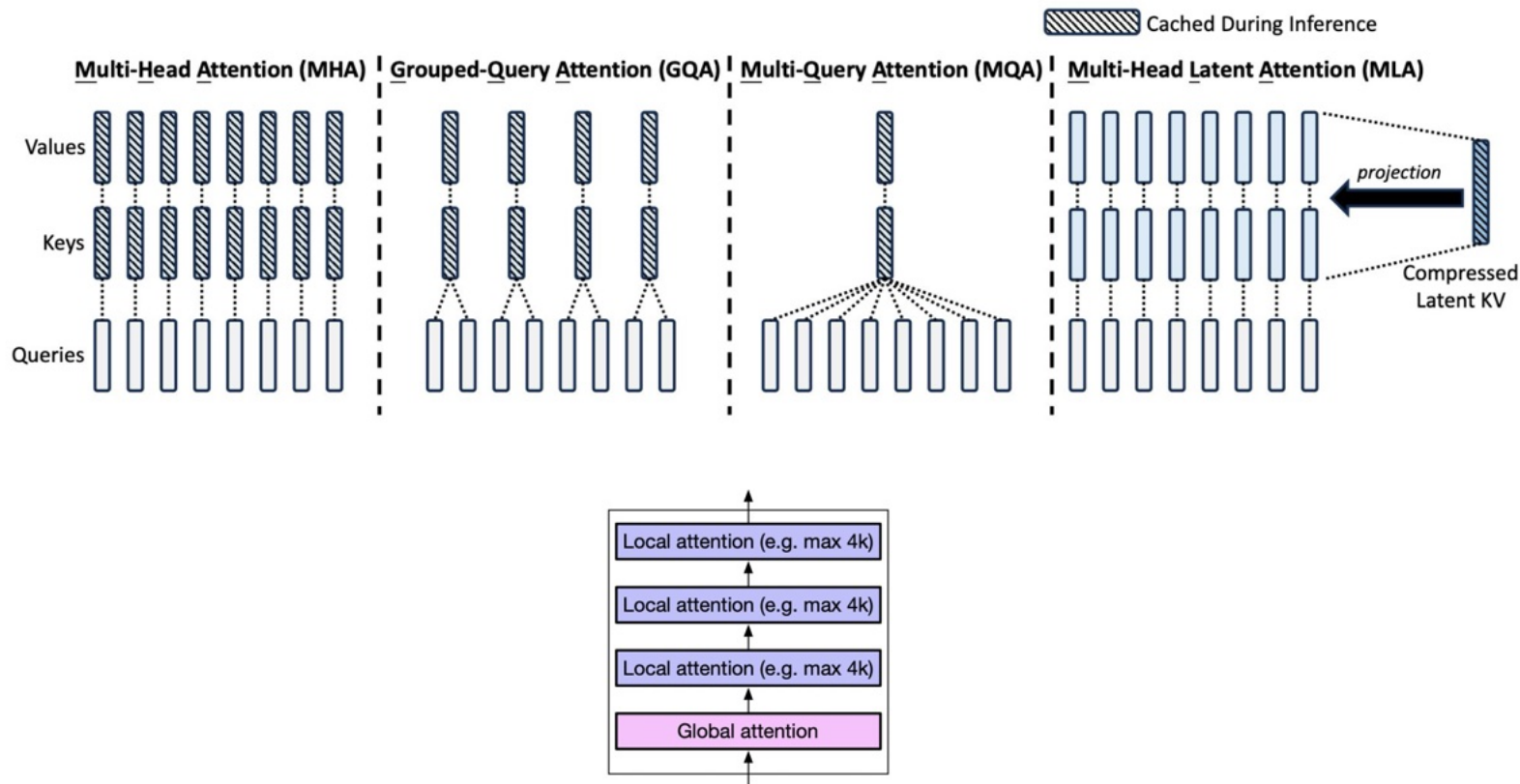
Table 3 Overview of the key hyperparameters of Llama 3. We display settings for 8B, 70B, and 405B language models.

Understand KV-caching. See also RoPE scaling, e.g. YaRN <https://arxiv.org/pdf/2309.00071>

Mixture of Experts (MoE) (e.g. DeepSeek V3)

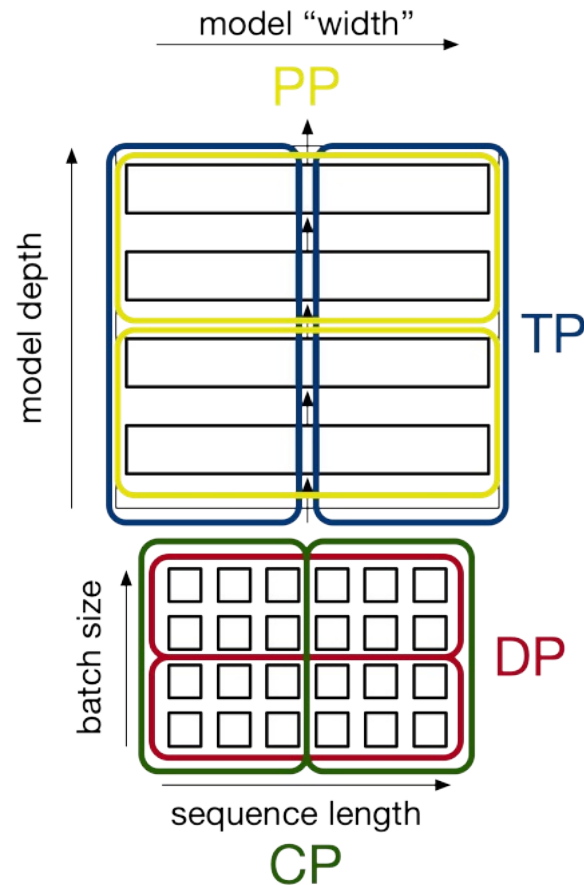


Memory efficient attention: GQA, MQA, MLA, local/global



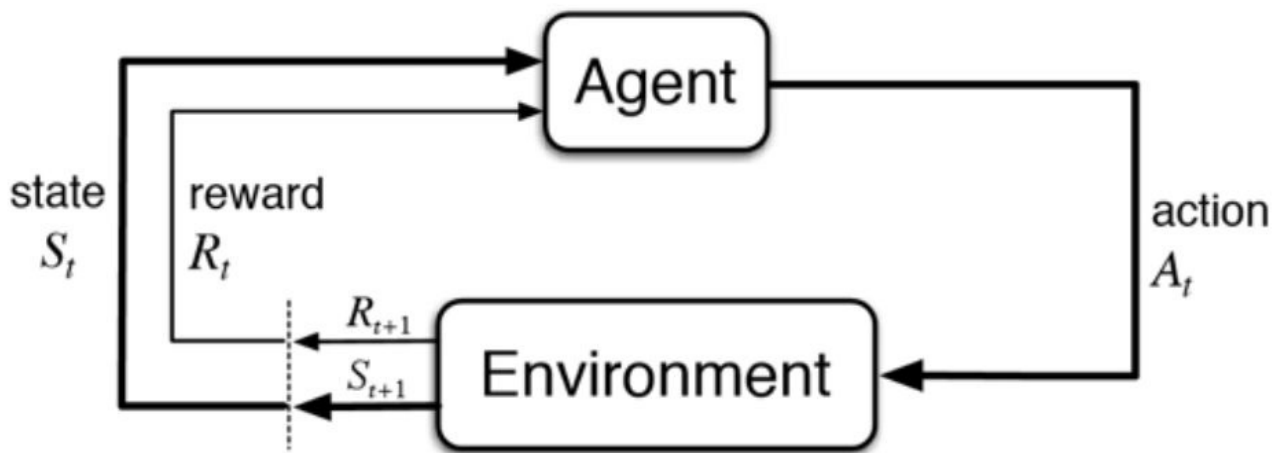
Types of parallelism

- Data Parallelism
(including Fully Sharded Data Parallelism)
- Tensor Parallelism
- Context Parallelism
- Pipeline Parallelism



Reinforcement Learning

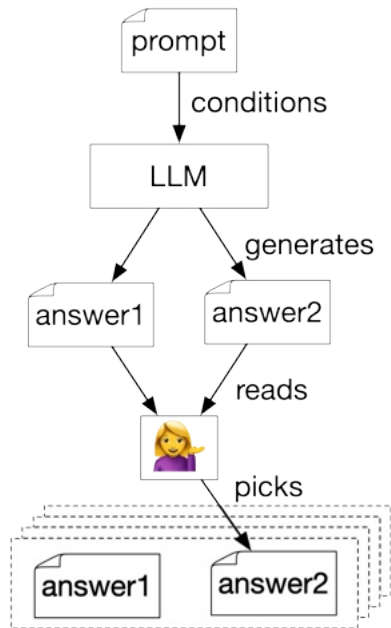
- Actions that you take influence what state and (scalar) rewards you get from the environment.



From Sutton & Barto

RLHF

Training data generation



Training

- Rejection sampling:
 - Cut off the logits head and replace with a regression => train a reward model on preference data
 - Use the LLM to generate lots of answers for the set of prompts in the instruct/alignment set => perform rejection sampling with the reward model
 - Train (supervisedly) on the generations that pass rejection sampling
- PPO (proximal policy optimization):
 - Same step 1
 - Train the LLM with PPO (policy gradient + clippings) on the reward model predictions
- DPO (*direct* policy optimization):
 - No step 1
 - Train the LLM to optimize directly “more like the picked answer, less like the unpicked answer” on the training set with a PG-like loss.

Grounding LLMs in Code Execution

Code generation: existing benchmarks

- Start to saturate
- Or go:
 - Agentic (e.g. SWEBench, ...)
 - **Competitive programming!**

EvalPlus base leaderboard (HumanEval / MBPP average)

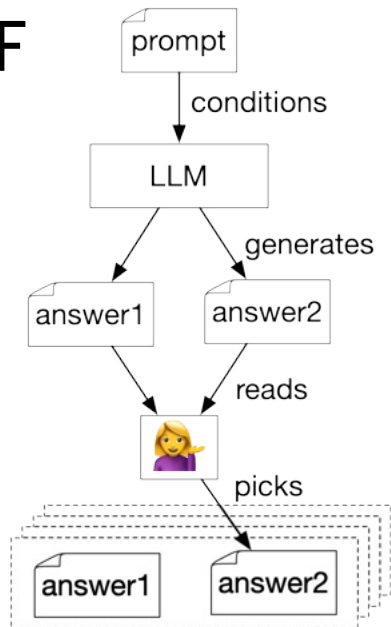
#	Model	pass@1
1	🏆 01 Preview (Sept 2024) 🌟	95.9
2	🥈 01 Mini (Sept 2024) 🌟	94.7
3	🥉 Qwen2.5-Coder-32B-Instruct 🌟	91.3
4	GPT 4o (Aug 2024) 🌟	90.2
5	Gemini 1.5 Pro 002 🌟	89.3
6	DeepSeek-V2.5 (Nov 2024) 🌟	88.9
7	Claude Sonnet 3.5 (June 2024) 🌟	88.3
8	DeepSeek-Coder-V2-Instruct 🌟	87.4
9	Grok Beta 🌟	87.2
10	GPT 4o Mini (July 2024) 🌟	86.9

RLEF: GROUNDING CODE LLMs IN EXECUTION FEEDBACK WITH REINFORCEMENT LEARNING

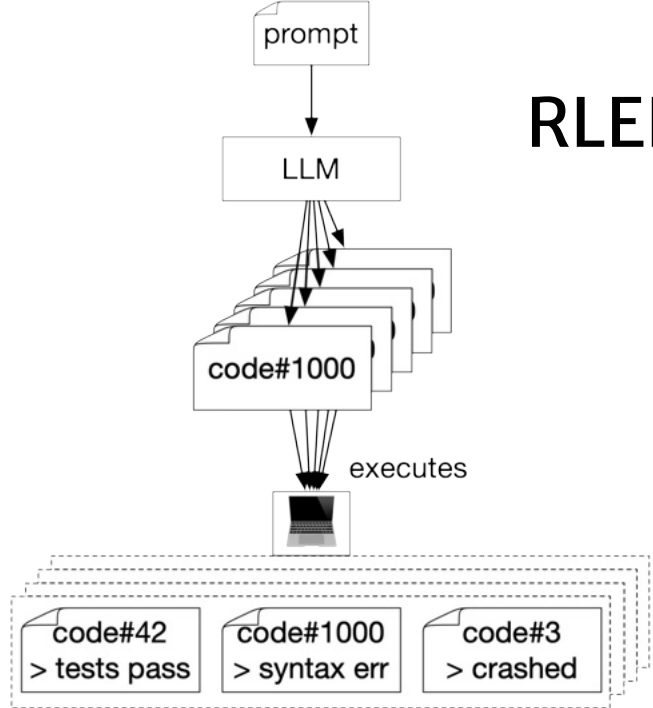
Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Taco Cohen, Gabriel Synnaeve
Meta AI (FAIR)
`{jgehring, gab}@meta.com`

RLHF

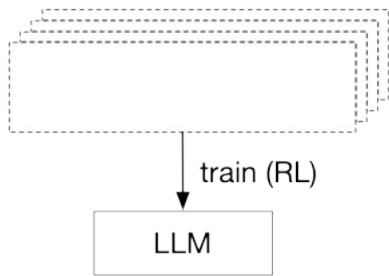
Training data generation



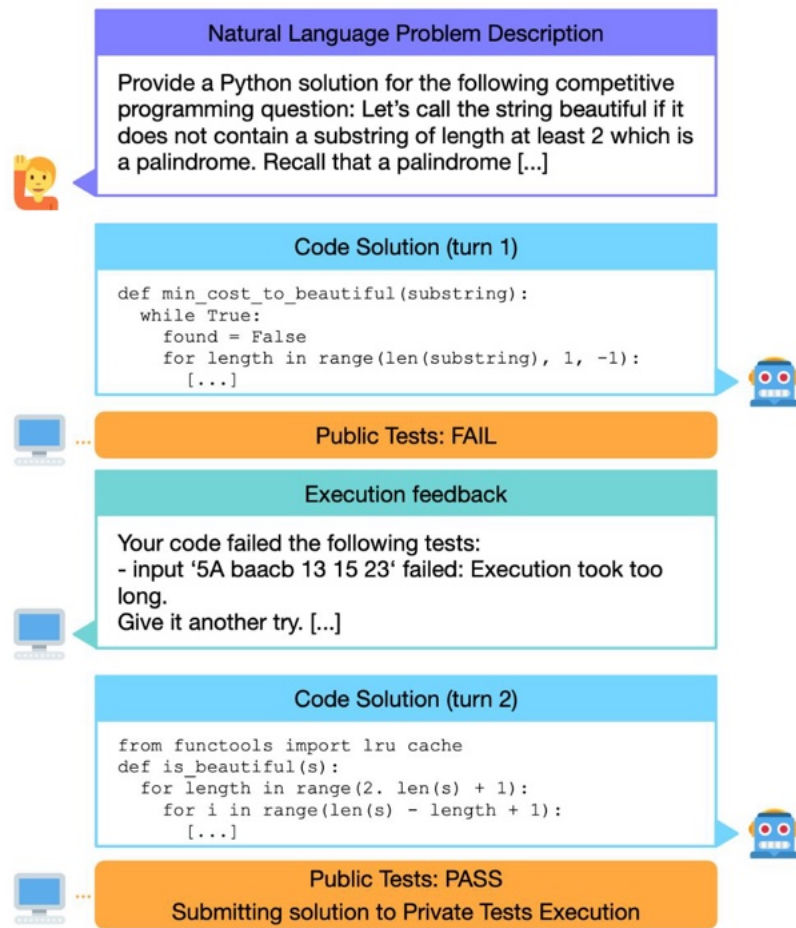
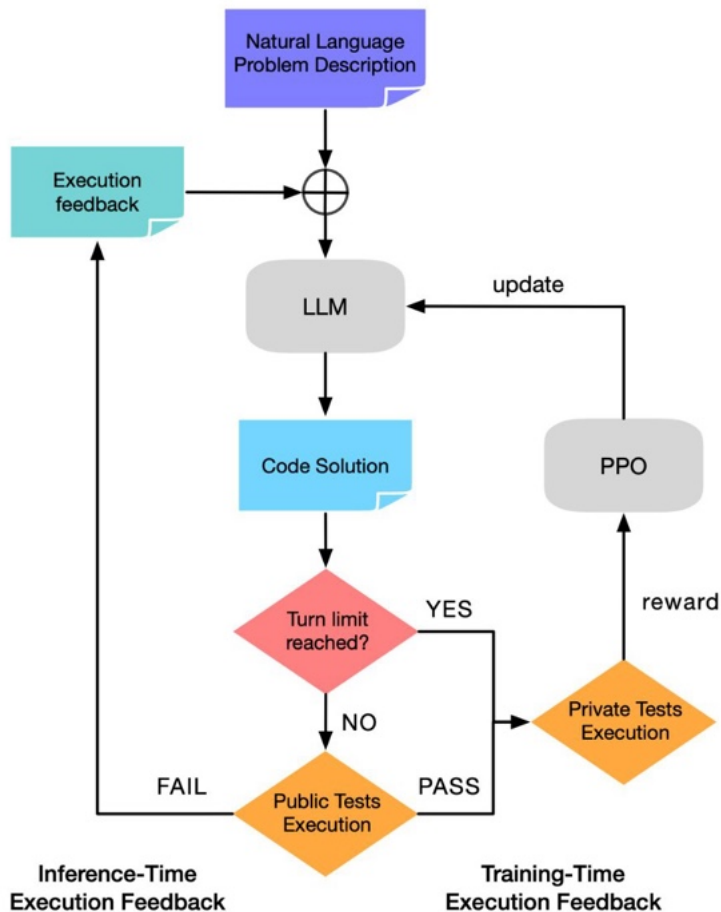
RLEF



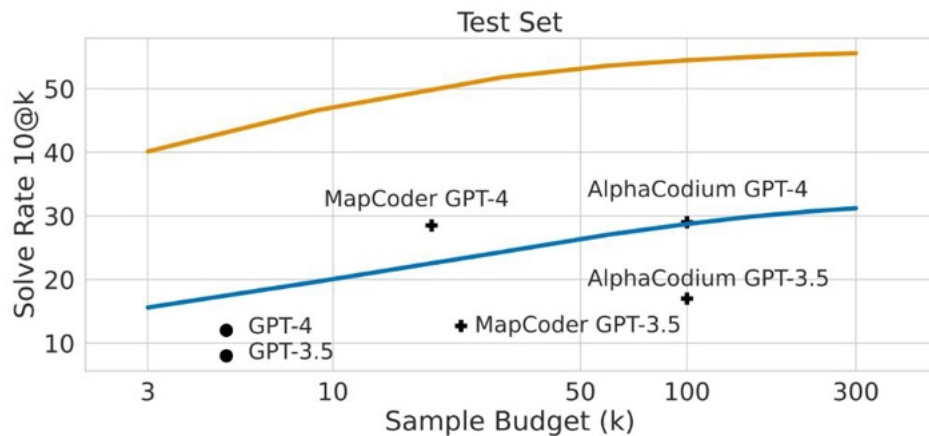
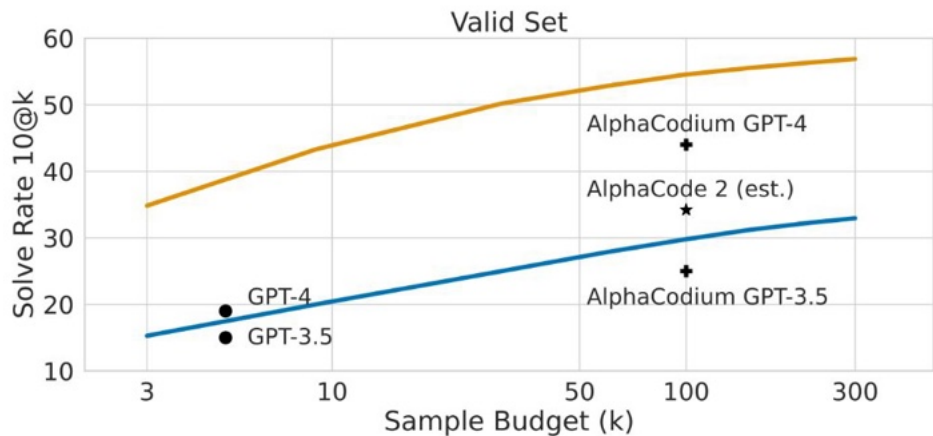
Training



CodeGen: RL with Execution Feedback (RLEF)



Results on DeepMind CodeContests (CodeForces)



— Llama 3.1 8B + RLEF — Llama 3.1 70B + RLEF • General Purpose + Prompt Engineering * Domain-Adapted

Model	Source	n@k	Valid Set	Test Set
AlphaCode 9B	Li et al. (2022)	10@1k	16.9	13.3
AlphaCode 41B + clustering	Li et al. (2022)	10@1k	21.0	16.4
Code Llama 34B + PPO	Xu et al. (2024)	10@1k	19.7	22.4
AlphaCodium gpt-3.5-turbo-16k	Ridnik et al. (2024)	5@100	25	17
AlphaCodium gpt-4-0613	Ridnik et al. (2024)	5@100	44	29
MapCoder gpt-3.5-turbo-1106	Islam et al. (2024)	1@23	-	12.7
MapCoder gpt-4-1106-preview	Islam et al. (2024)	1@19	-	28.5
Llama 3.0 8B Instruct	Ours	1@3	4.1	3.2
+ RLEF	Ours	1@3	12.5	12.1
Llama 3.1 8B Instruct	Ours	1@3	8.9	10.5
+ RLEF	Ours	1@3	17.2	16.0
Llama 3.1 70B Instruct	Ours	1@3	25.9	27.5
+ RLEF	Ours	1@3	37.5	40.1
Llama 3.1 8B Instruct	Ours	10@100	21.7	24.8
+ RLEF	Ours	10@100	29.8	28.7
Llama 3.1 70B Instruct	Ours	10@100	50.2	50.3
+ RLEF	Ours	10@100	54.5	54.5

Execution feedback  Reward

$$c_t = o_0, a_0, o_1, a_1, \dots, o_t$$

$$R(s_t, a_t) = r(s_t, a_t) - \beta \log \frac{\pi(a_t|c_t)}{\rho(a_t|c_t)}, \quad r(s_t, a_t) = \begin{cases} 1, & \text{if end of episode and all tests pass} \\ -1, & \text{if end of episode and any test fails} \\ -0.2, & \text{if } a_t \text{ does not contain valid code} \end{cases}$$

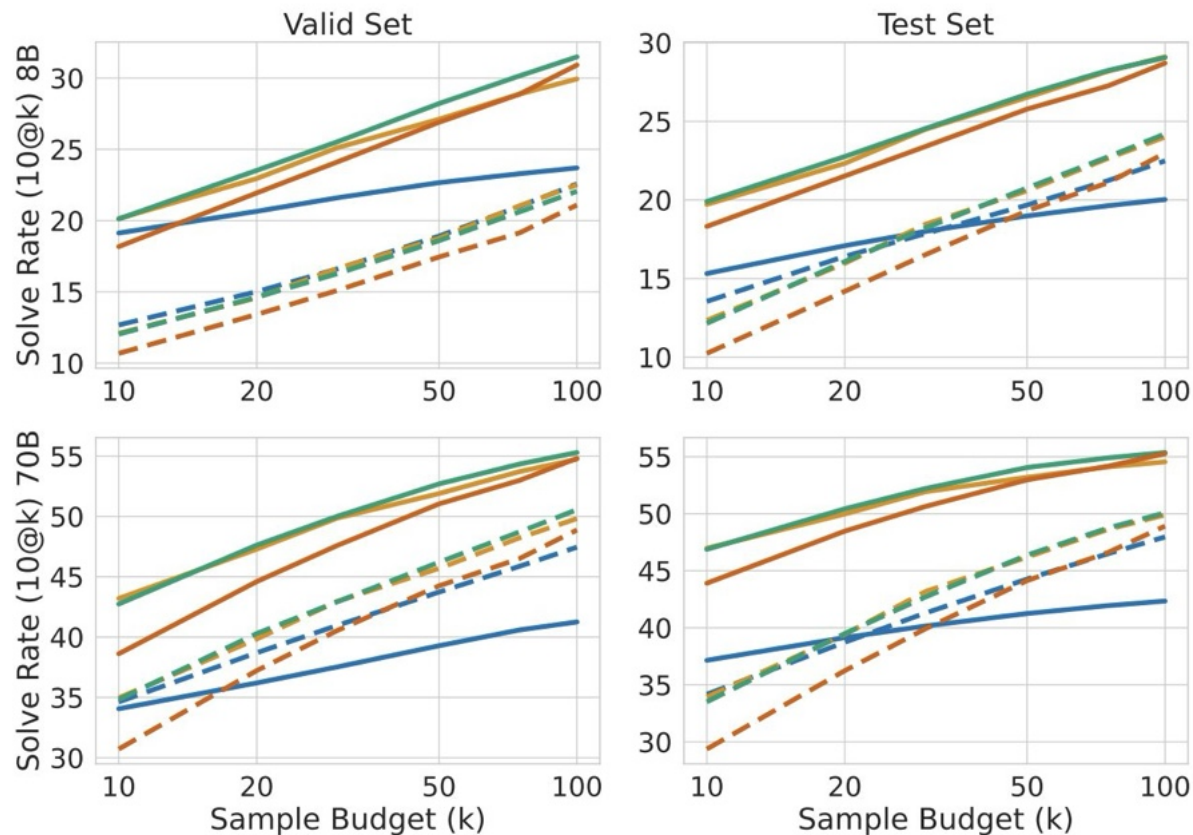
$$A_t = R(s_t, a_t) - V(c_t)$$

RLEF vs. SFT

Model	Method	Valid	Test
8B Instruct	–	8.9	10.5
	Few-Shot	8.5	8.5
	SFT	10.3	10.0
	RLEF	17.2	16.0
70B Instruct	–	25.9	27.5
	Few-Shot	22.5	20.3
	SFT	27.7	27.2
	RLEF	37.5	40.1

RLEF: importance of multiturn

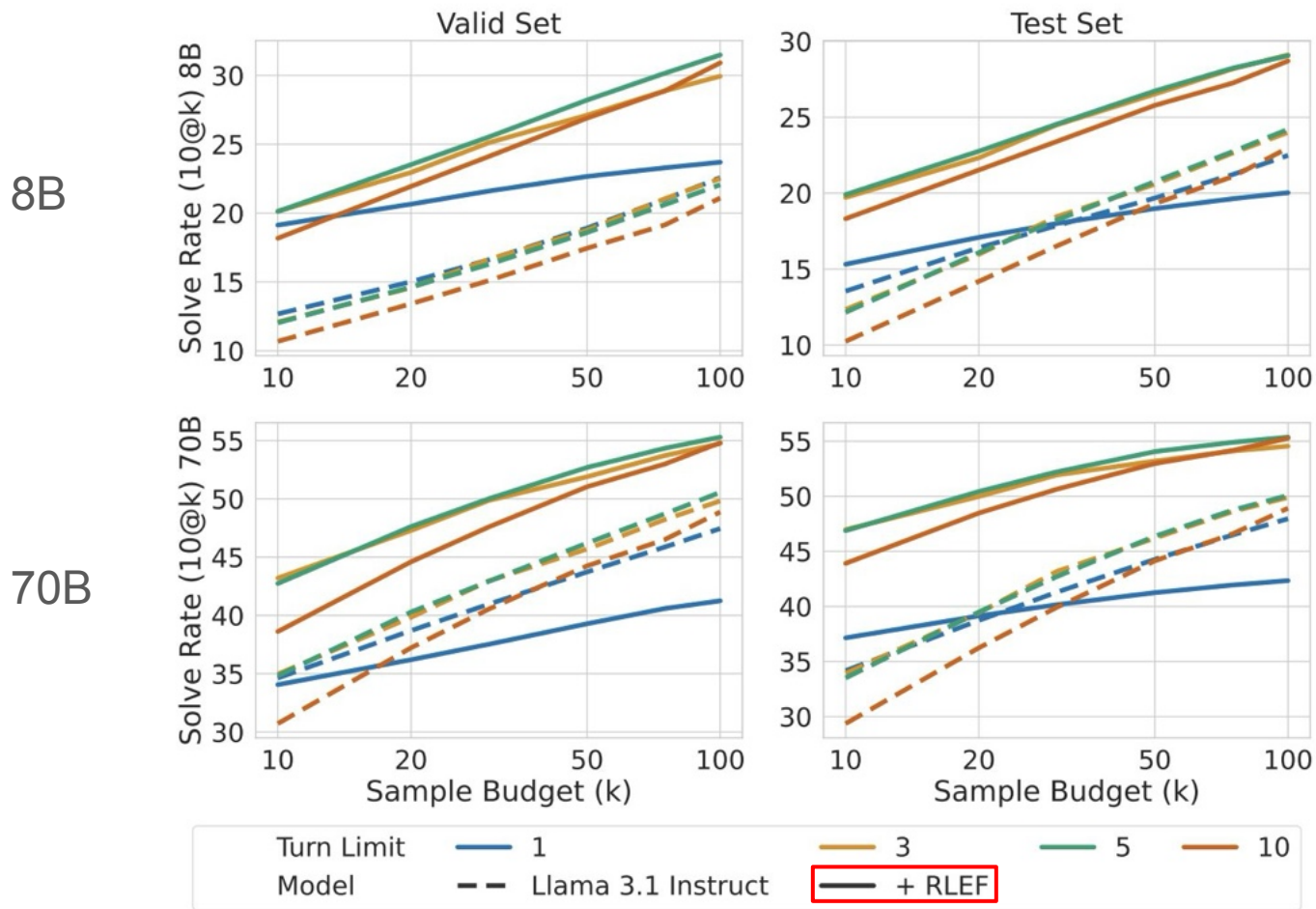
8B



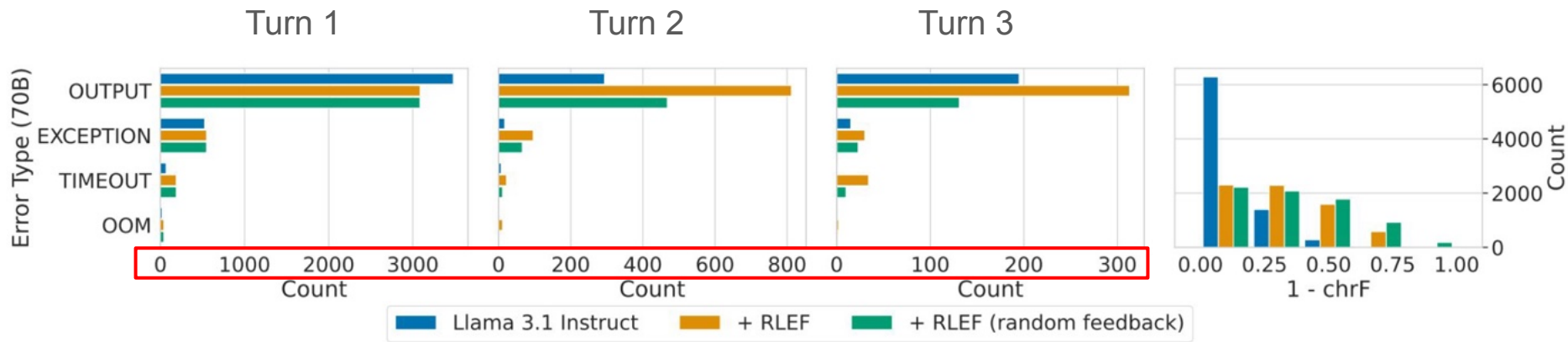
70B

Model	Training	Valid	
		ST	MT
8B \ Instruct	–	9.4	8.9
	ST	10.3	10.2
	MT	16.2	17.2
70B \ Instruct	–	25.6	25.9
	ST	28.3	31.1
	MT	25.8	37.5

RLEF: importance of multiturn



RLEF: what does it learn?

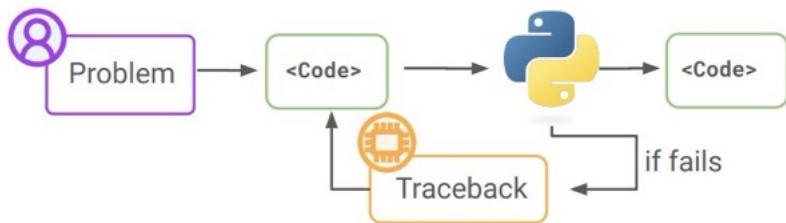


WHAT MAKES LARGE LANGUAGE MODELS REASON IN (MULTI-TURN) CODE GENERATION?

**Kunhao Zheng^{1,2,*}, Juliette Decugis^{1,*}, Jonas Gehring¹, Taco Cohen¹,
Benjamin Negrevergne², Gabriel Synnaeve¹**

¹Meta AI (FAIR), ²Paris Dauphine University - PSL

{kunhao, jdecugis, gab}@meta.com

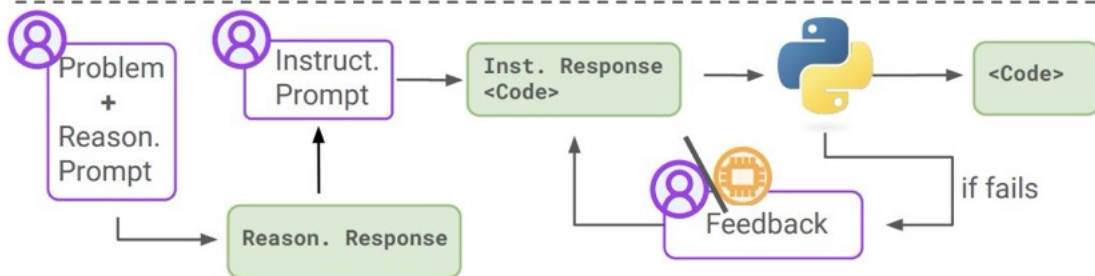


Code

```

n, m = map(int, input().split())
words = [input().strip() for _ in range(n)]
words.sort(key=lambda x: [(-ord(s[i-1]), ord(s[i])) if i % 2 else (ord(s[i-1]), -ord(s[i])) for i in range(1, m+1)])
...

```



Reason. Response

```

def custom_sort(s):
    # Custom sorting key for asc-desc-ending order.
    This function returns a tuple where the first element is a
    tuple of characters at odd indices in ascending order and
    the second element is a tuple of characters at even
    indices in descending order.

```

Problem Statement

Andi and Budi were given an assignment to tidy up their bookshelf of n books. Each book is represented by the book title — a string s_i numbered from 1 to n , each with length m . Andi really wants to sort the book lexicographically ascending, while Budi wants to sort it lexicographically descending. Settling their fight, they decided to combine their idea and sort it asc-desc-endingly.

...

Example

Input

```
5 2\nAA\nAB\nBB\nBA\nAZ
```

Output

```
5 2 1 3 4
```

...

Reason. Prompt

Write helper functions docstring and explain.

Inst. Prompt

Write comment before each line.

Execution Feedback

Your code failed the following tests: - input '5 2\nAA\nAB\nBB\nBA\nAZ'

Traceback:

...

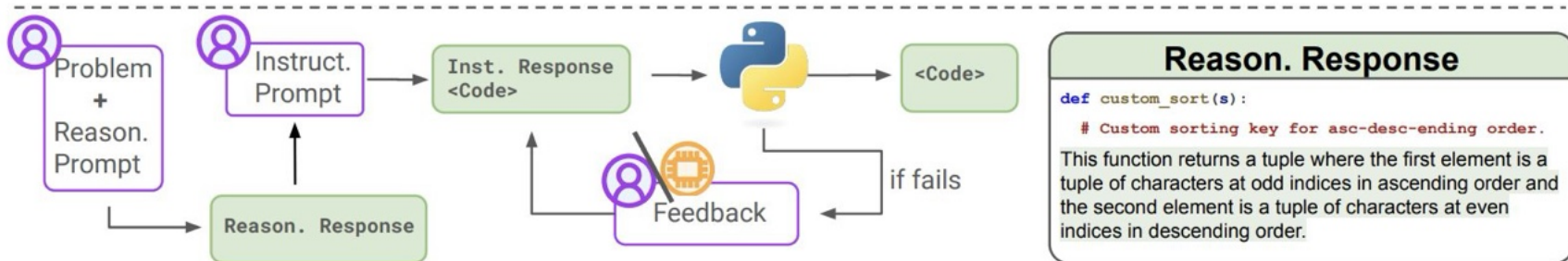
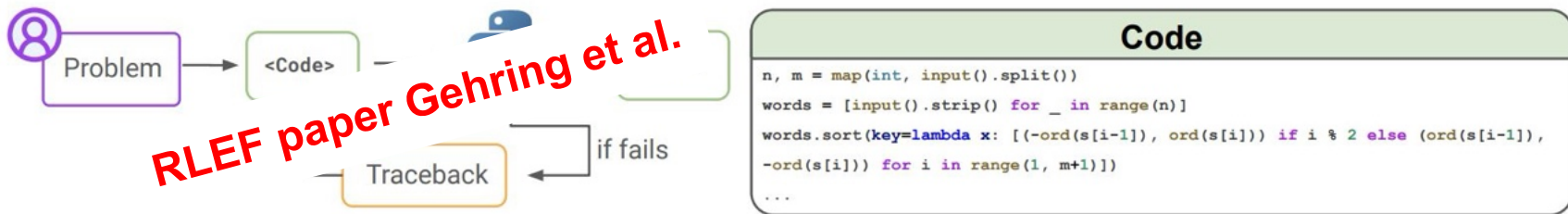
Your code is wrong.

Inst. Response

```

def custom_sort(s):
    ...
def main():
    # Read the number of book titles (n) and the
    # length of each title (m)
    n, m = map(int, input().strip().split())
    # Read each book title and store it
    strings = [input().strip() for _ in range(n)]
    # Sort the book titles based on the
    # "asc-desc-endingly" criteria
    ...

```



Problem Statement

Andi and Budi were given an assignment to tidy up their bookshelf of n books. Each book is represented by the book title — a string s_i numbered from 1 to n , each with length m . Andi really wants to sort the book lexicographically ascending, while Budi wants to sort it lexicographically descending. Settling their fight, they decided to combine their idea and sort it asc-desc-endingly.

...

Example

Input

```
5 2\nAA\nAB\nBB\nBA\nAZ
```

Output

```
5 2 1 3 4
```

...

Reason. Prompt

Write helper functions docstring and explain.

Inst. Prompt

Write comment before each line.

Execution Feedback

Your code failed the following tests: - input '5 2\nAA\nAB\nBB\nBA\nAZ'

Traceback:

...

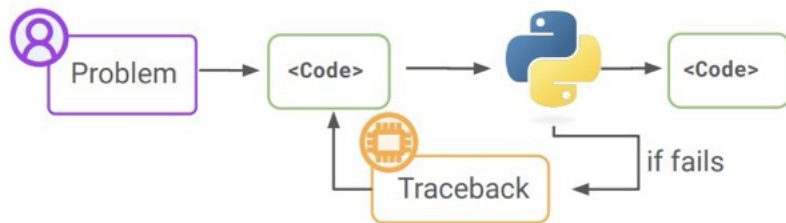
Your code is wrong.

Inst. Response

```

def custom_sort(s):
    ...
def main():
    # Read the number of book titles (n) and the
    # length of each title (m)
    n, m = map(int, input().strip().split())
    # Read each book title and store it
    strings = [input().strip() for _ in range(n)]
    # Sort the book titles based on the
    # "asc-desc-endingly" criteria
    ...

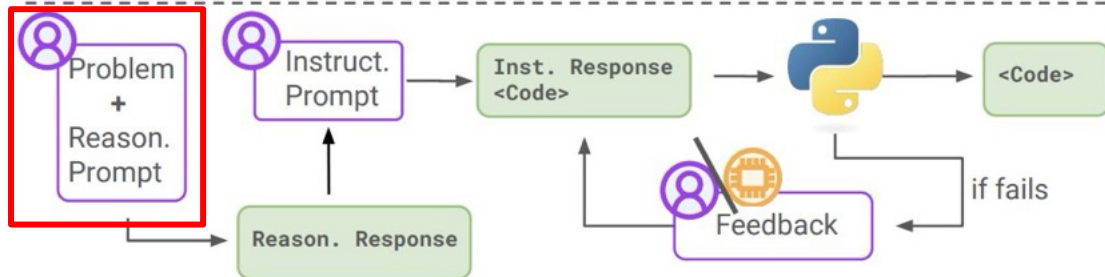
```



Code

```

n, m = map(int, input().split())
words = [input().strip() for _ in range(n)]
words.sort(key=lambda x: [(-ord(s[i-1]), ord(s[i])) if i % 2 else (ord(s[i-1]), -ord(s[i])) for i in range(1, m+1)])
...
  
```



Reason. Response

```

def custom_sort(s):
    # Custom sorting key for asc-desc-ending order.
    This function returns a tuple where the first element is a
    tuple of characters at odd indices in ascending order and
    the second element is a tuple of characters at even
    indices in descending order.
  
```

Problem Statement

Andi and Budi were given an assignment to tidy up their bookshelf of n books. Each book is represented by the book title — a string s_i numbered from 1 to n , each with length m . Andi really wants to sort the book lexicographically ascending, while Budi wants to sort it lexicographically descending. Settling their fight, they decided to combine their idea and sort it asc-desc-endingly.

...

Example

Input

```
5 2\nAA\nAB\nBB\nBA\nAZ
```

Output

```
5 2 1 3 4
```

...

Reason. Prompt

Write helper functions docstring and explain.

Inst. Prompt

Write comment before each line.

Execution Feedback

Your code failed the following tests: - input '5 2\nAA\nAB\nBB\nBA\nAZ'

Traceback:

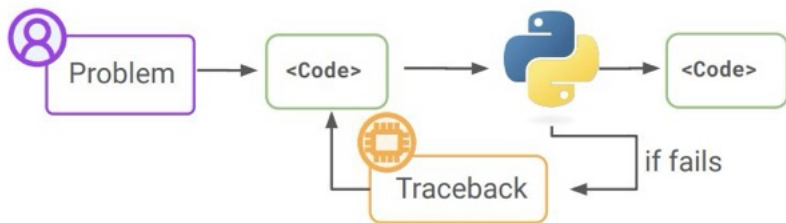
...

Your code is wrong.

Inst. Response

```

def custom_sort(s):
    ...
def main():
    # Read the number of book titles (n) and the
    # length of each title (m)
    n, m = map(int, input().strip().split())
    # Read each book title and store it
    strings = [input().strip() for _ in range(n)]
    # Sort the book titles based on the
    # "asc-desc-endingly" criteria
    ...
  
```

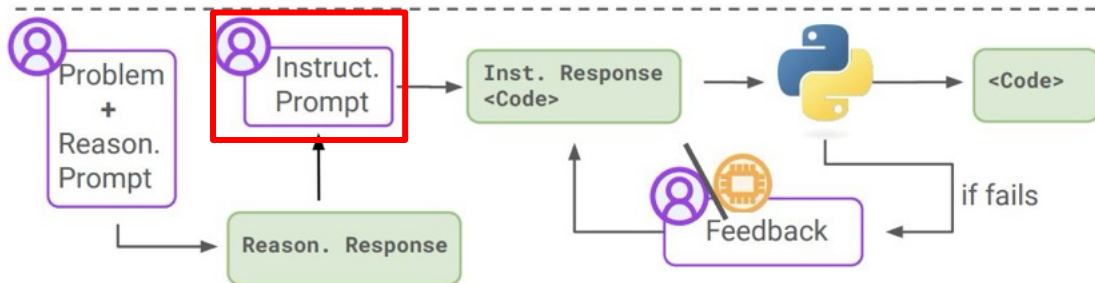



Code

```

n, m = map(int, input().split())
words = [input().strip() for _ in range(n)]
words.sort(key=lambda x: [(-ord(s[i-1]), ord(s[i])) if i % 2 else (ord(s[i-1]), -ord(s[i])) for i in range(1, m+1)])
...

```



Reason. Response

```

def custom_sort(s):
    # Custom sorting key for asc-desc-ending order.
    This function returns a tuple where the first element is a
    tuple of characters at odd indices in ascending order and
    the second element is a tuple of characters at even
    indices in descending order.

```

Problem Statement

Andi and Budi were given an assignment to tidy up their bookshelf of n books. Each book is represented by the book title — a string s_i numbered from 1 to n , each with length m . Andi really wants to sort the book lexicographically ascending, while Budi wants to sort it lexicographically descending. Settling their fight, they decided to combine their idea and sort it asc-desc-endingly.

...

Example

Input

```
5 2\nAA\nAB\nBB\nBA\nAZ
```

Output

```
5 2 1 3 4
```

...

Reason. Prompt

Write helper functions docstring and explain.

Inst. Prompt

Write comment before each line.

Execution Feedback

Your code failed the following tests: - input '5 2\nAA\nAB\nBB\nBA\nAZ'

Traceback:

...

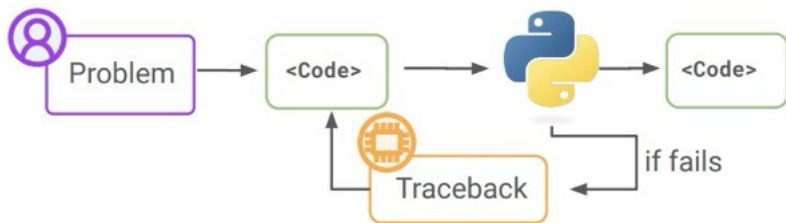
Your code is wrong.

Inst. Response

```

def custom_sort(s):
    ...
def main():
    # Read the number of book titles (n) and the
    # length of each title (m)
    n, m = map(int, input().strip().split())
    # Read each book title and store it
    strings = [input().strip() for _ in range(n)]
    # Sort the book titles based on the
    # "asc-desc-endingly" criteria
    ...

```

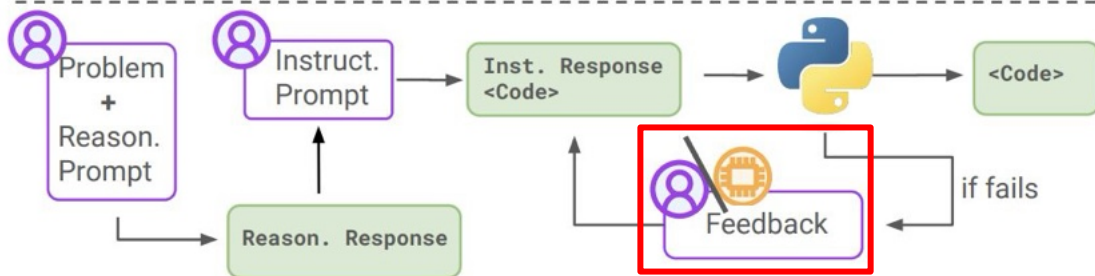


Code

```

n, m = map(int, input().split())
words = [input().strip() for _ in range(n)]
words.sort(key=lambda x: [(-ord(s[i-1]), ord(s[i])) if i % 2 else (ord(s[i-1]), -ord(s[i])) for i in range(1, m+1)])
...

```



Reason. Response

```

def custom_sort(s):
    # Custom sorting key for asc-desc-ending order.
    This function returns a tuple where the first element is a
    tuple of characters at odd indices in ascending order and
    the second element is a tuple of characters at even
    indices in descending order.

```

Problem Statement

Andi and Budi were given an assignment to tidy up their bookshelf of n books. Each book is represented by the book title — a string s_i numbered from 1 to n , each with length m . Andi really wants to sort the book lexicographically ascending, while Budi wants to sort it lexicographically descending. Settling their fight, they decided to combine their idea and sort it asc-desc-endingly.

...

Example

Input

```
5 2\nAA\nAB\nBB\nBA\nAZ
```

Output

```
5 2 1 3 4
```

...

Reason. Prompt

Write helper functions docstring and explain.

Inst. Prompt

Write comment before each line.

Execution Feedback

Your code failed the following tests: - input '5 2\nAA\nAB\nBB\nBA\nAZ'

Traceback:

...

Your code is wrong.

Inst. Response

```

def custom_sort(s):
    ...
def main():
    # Read the number of book titles (n) and the
    # length of each title (m)
    n, m = map(int, input().strip().split())
    # Read each book title and store it
    strings = [input().strip() for _ in range(n)]
    # Sort the book titles based on the
    # "asc-desc-endingly" criteria
    ...

```

Prompting space we explored

SINGLE-TURN

Reasoning Prompts (NL $\rightarrow$ NL)

CoA [1], Self-discover [2],
Decomposed prompting [3]

Instruction Prompts (NL $\rightarrow$ Code)

Parsel [4], LLM assisted code
cleaning [5]

Natural Language

Explain input output pairs

Rate the problem difficulty

Write a solution in natural
language.

Reflect on the problem, and
describe it in your own words

...

Code

Generate a naive solution.

Propose a clever algorithm.

Describe the necessary
intermediate variables.

Write the docstring and
signatures of helper functions

...

MULTI-TURN

Feedback Prompts

Learn to Debug (LDB) [6], NeXT [7], Self-Debugging [8],
CodeChain [9], Self-refine [10], REFINER [11], CodeRepair [12],
General to Specific Prompting [13]

Execution Feedback

Your code is wrong.

You failed tests. Expected: 5, got: 2

You failed the tests: for input 1 expected: 5 got: 2
You passed some tests: for input 0 expected: 8 and got: 8

Here are intermediate values, analyze and fix your code.

...

Model	Variants	CodeContests / Test			
		1@3	10@30	33@100	100@300
Llama 3.0 8B		2.9	8.0	12.6	-
	+ CoT	3.4 _{+0.5}	11.7 _{+3.7}	17.3 _{+4.7}	-
	+ Multi-turn	2.4 _{-0.5}	8.0 _{+0.0}	12.8 _{+0.2}	16.7
	+ Multi-turn CoT	2.8 _{-0.1}	9.8 _{+1.8}	14.9 _{+2.3}	19.4
Llama 3.0 70B		9.6	18.9	23.1	-
	+ CoT	10.4 _{+0.8}	26.0 _{+7.1}	33.0 _{+9.9}	-
	+ Multi-turn	10.1 _{+0.5}	21.0 _{+2.1}	26.7 _{+3.6}	32.7
	+ Multi-turn CoT	11.1 _{+1.5}	26.5 _{+7.6}	34.3 _{+11.2}	40.4
Llama 3.1 8B		7.7	18.2	23.8	-
	+ CoT	8.0 _{+0.3}	19.5 _{+1.3}	26.1 _{+2.3}	-
	+ Multi-turn	7.0 _{-0.7}	18.8 _{+0.6}	24.5 _{+0.7}	30.4
	+ Multi-turn CoT	6.9 _{-0.8}	19.4 _{+1.2}	26.0 _{+2.2}	31.5
Llama 3.1 70B		24.1	42.3	49.8	-
	+ CoT	26.4 _{+2.3}	47.8 _{+5.5}	54.8 _{+5.0}	-
	+ Multi-turn	24.1 _{+0.0}	43.8 _{+1.5}	51.6 _{+1.8}	56.2
	+ Multi-turn CoT	27.7 _{+3.6}	48.4 _{+6.1}	55.3 _{+5.5}	59.6

Model	Variants	CodeContests / Test			
		1@3	10@30	33@100	100@300
Llama 3.0 8B		2.9	8.0	12.6	-
	+ CoT	3.4 _{+0.5}	11.7 _{+3.7}	17.3_{+4.7}	-
	+ Multi-turn	2.4 _{-0.5}	8.0 _{+0.0}	12.8 _{+0.2}	16.7
	+ Multi-turn CoT	2.8 _{-0.1}	9.8 _{+1.8}	14.9 _{+2.3}	19.4
Llama 3.0 70B		9.6	18.9	23.1	-
	+ CoT	10.4 _{+0.8}	26.0 _{+7.1}	33.0 _{+9.9}	-
	+ Multi-turn	10.1 _{+0.5}	21.0 _{+2.1}	26.7 _{+3.6}	32.7
	+ Multi-turn CoT	11.1 _{+1.5}	26.5 _{+7.6}	34.3_{+11.2}	40.4
Llama 3.1 8B		7.7	18.2	23.8	-
	+ CoT	8.0 _{+0.3}	19.5 _{+1.3}	26.1_{+2.3}	-
	+ Multi-turn	7.0 _{-0.7}	18.8 _{+0.6}	24.5 _{+0.7}	30.4
	+ Multi-turn CoT	6.9 _{-0.8}	19.4 _{+1.2}	26.0 _{+2.2}	31.5
Llama 3.1 70B		24.1	42.3	49.8	-
	+ CoT	26.4 _{+2.3}	47.8 _{+5.5}	54.8 _{+5.0}	-
	+ Multi-turn	24.1 _{+0.0}	43.8 _{+1.5}	51.6 _{+1.8}	56.2
	+ Multi-turn CoT	27.7 _{+3.6}	48.4 _{+6.1}	55.3_{+5.5}	59.6

Model	Variants	CodeContests / Test			
		1@3	10@30	33@100	100@300
Llama 3.0 8B		2.9	8.0	12.6	-
	+ CoT	3.4 _{+0.5}	11.7 _{+3.7}	17.3 _{+4.7}	-
	+ Multi-turn	2.4 _{-0.5}	8.0 _{+0.0}	12.8 _{+0.2}	16.7
	+ Multi-turn CoT	2.8 _{-0.1}	9.8 _{+1.8}	14.9 _{+2.3}	19.4
Llama 3.0 70B		9.6	18.9	23.1	-
	+ CoT	10.4 _{+0.8}	26.0 _{+7.1}	33.0 _{+9.9}	-
	+ Multi-turn	10.1 _{+0.5}	21.0 _{+2.1}	26.7 _{+3.6}	32.7
	+ Multi-turn CoT	11.1 _{+1.5}	26.5 _{+7.6}	34.3 _{+11.2}	40.4
Llama 3.1 8B		7.7	18.2	23.8	-
	+ CoT	8.0 _{+0.3}	19.5 _{+1.3}	26.1 _{+2.3}	-
	+ Multi-turn	7.0 _{-0.7}	18.8 _{+0.6}	24.5 _{+0.7}	30.4
	+ Multi-turn CoT	6.9 _{-0.8}	19.4 _{+1.2}	26.0 _{+2.2}	31.5
Llama 3.1 70B		24.1	42.3	49.8	-
	+ CoT	26.4 _{+2.3}	47.8 _{+5.5}	54.8 _{+5.0}	-
	+ Multi-turn	24.1 _{+0.0}	43.8 _{+1.5}	51.6 _{+1.8}	56.2
	+ Multi-turn CoT	27.7 _{+3.6}	48.4 _{+6.1}	55.3 _{+5.5}	59.6
Llama 3.1 70B ^{RFT}		26.2	45.1	50.9	-
	+ Multi-turn	29.7 _{+3.5}	50.5 _{+5.4}	57.2 _{+6.3}	61.1

CoT across models

Variants	GPT-4o		Llama 3.1 70B		Llama 3.1 405B	
	1@1	1@3	1@1	1@3	1@1	1@3
Single-turn	17.0	27.6	23.2	27.3	27.8	32.9
+ CoT	25.5 _{+8.5}	29.0 _{+1.4}	25.5 _{+2.3}	28.9 _{+1.6}	25.1 _{-2.7}	31.8 _{-1.1}
+ Multi-turn	-	23.1 _{-4.5}	-	29.5 _{+2.2}	-	35.4 _{+2.5}
+ Multi-turn CoT	-	31.5_{+3.9}	-	31.5_{+4.2}	-	40.1_{+7.2}

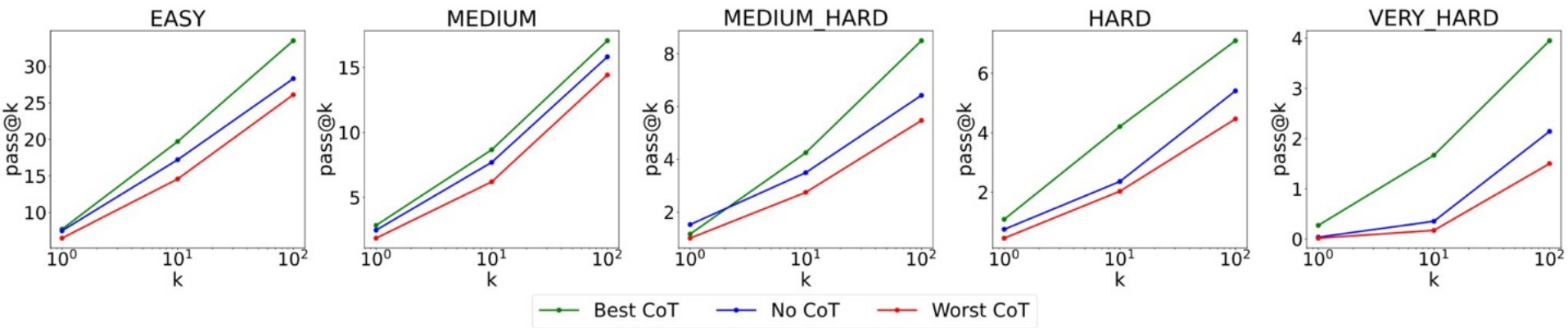
Ablating CoT prompts and CoT types of prompts

(we did a big “grid” search of 63 combinations, 8 *reasoning*, 6 *instruction*)

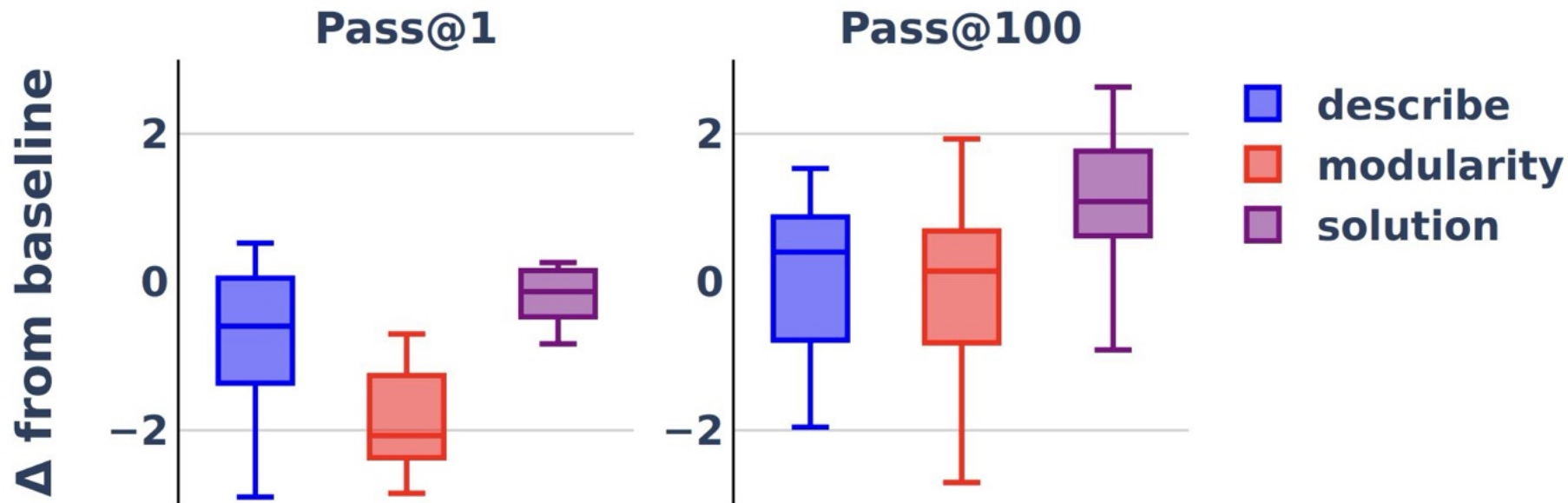
	Llama 3.0 8B		Llama 3.0 70B		Llama 3.1 8B		Llama 3.1 70B	
	pass@1	pass@100	pass@1	pass@100	pass@1	pass@100	pass@1	pass@100
Baseline	1.6	12.3	3.8	23.8	3.8	22.8	16.7	48.9
Worst <i>reasoning</i>	1.4	12.9	5.7	21.8	4.0	23.4	15.6	47.4
Worst <i>instruction</i>	1.4	11.3	3.4	25.1	3.7	20.9	14.9	48.4
Worst Combination	1.4	11.8	5.6	21.0	2.9	21.1	13.2	43.5
Best <i>reasoning</i>	1.8	15.7	7.0	30.4	4.1	25.7	<u>15.7</u>	52.2
Best <i>instruction</i>	<u>1.3</u>	13.5	5.5	29.6	<u>3.6</u>	24.6	16.8	53.8
Best Combination	<u>1.5</u>	17.3	5.3	33.1	4.0	26.1	<u>16.1</u>	54.1

=> Combining *reasoning* and *instruction* prompts works best

CoT helps most on hard examples

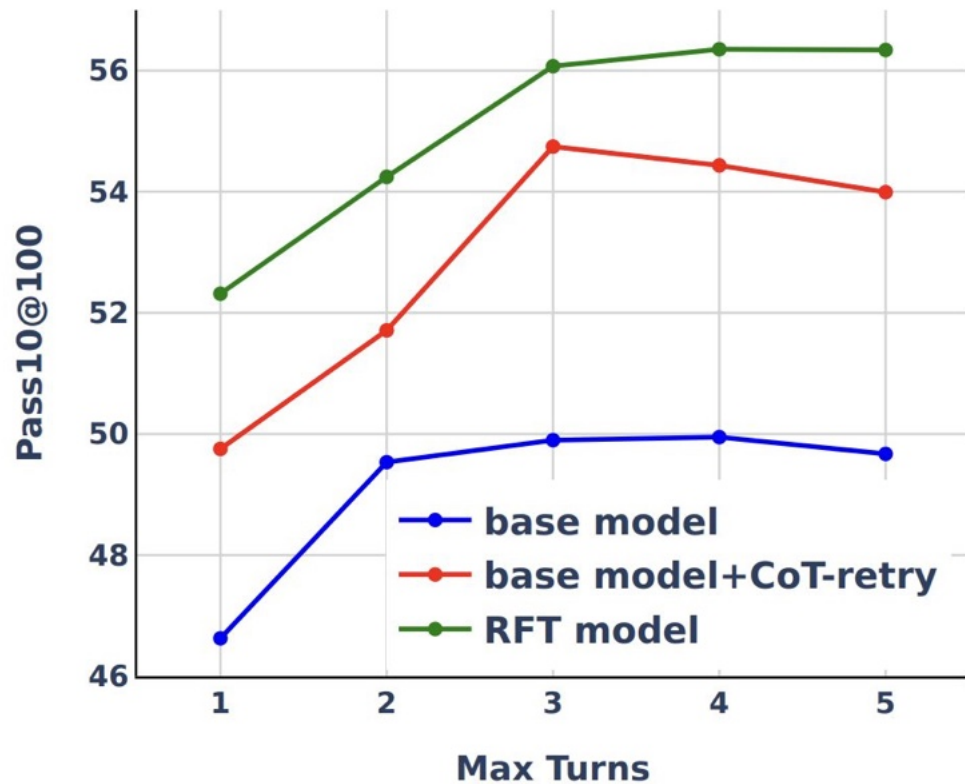


Solution-based *instruction* prompts work best for Llama 3.1



=> Alas: **prompt efficacy is model and sample size dependent**

Reasoning and execution feedback prompts enhance multi-turn performance



(But *reasoning* prompts are not additive)

Training (rejection sampling finetuning on Llama 3.1 70B)

Train on 160k multi-turn trajectories (285M tokens)

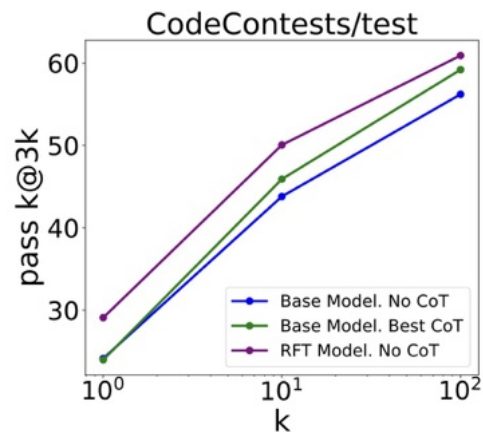
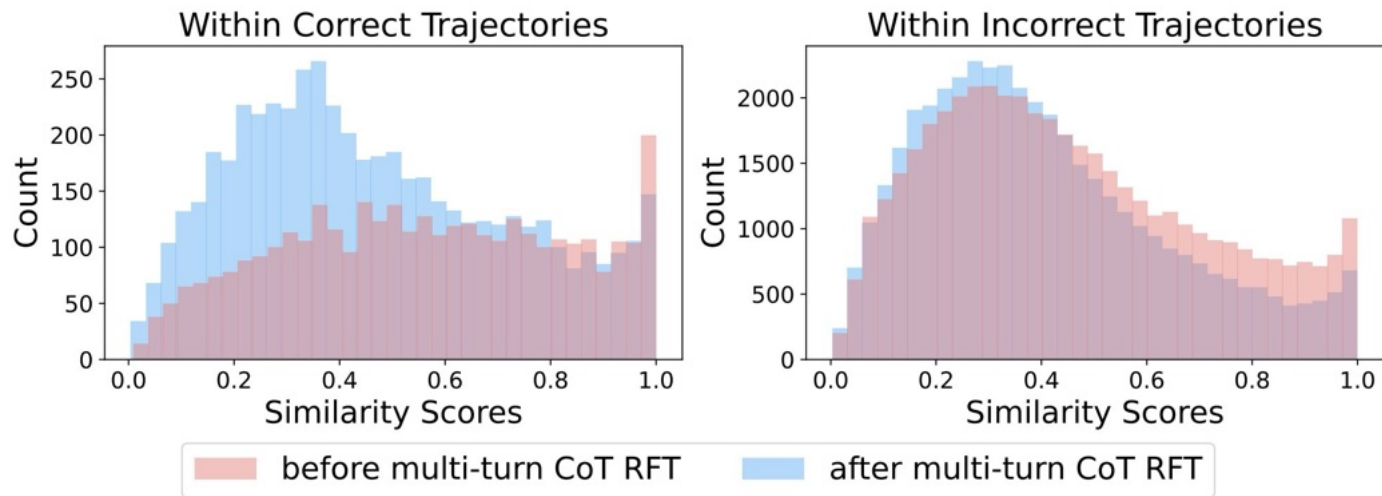
Model	easy		medium		medium_hard		hard		very_hard	
	1@3	100@300	1@3	100@300	1@3	100@300	1@3	100@300	1@3	100@300
Llama 3.1 70B	31.6	60.2	14.2	44.6	9.5	36.2	4.4	20.6	1.8	9.0
+ Multi-turn CoT	32.3	59.8	15.0	46.2	10.8	38.5	5.8	22.8	2.6	11.8
Llama 3.1 70B ^{RFT}	34.1	58.9	18.0	45.3	13.0	39.4	8.1	23.3	3.5	12.0

Training (rejection sampling finetuning on Llama 3.1 70B)

Train on 160k multi-turn trajectories (285M tokens)

Model	easy		medium		medium_hard		hard		very_hard	
	1@3	100@300	1@3	100@300	1@3	100@300	1@3	100@300	1@3	100@300
Llama 3.1 70B	31.6	60.2	14.2	44.6	9.5	36.2	4.4	20.6	1.8	9.0
+ Multi-turn CoT	32.3	59.8	15.0	46.2	10.8	38.5	5.8	22.8	2.6	11.8
Llama 3.1 70B ^{RFT}	34.1	58.9	18.0	45.3	13.0	39.4	8.1	23.3	3.5	12.0

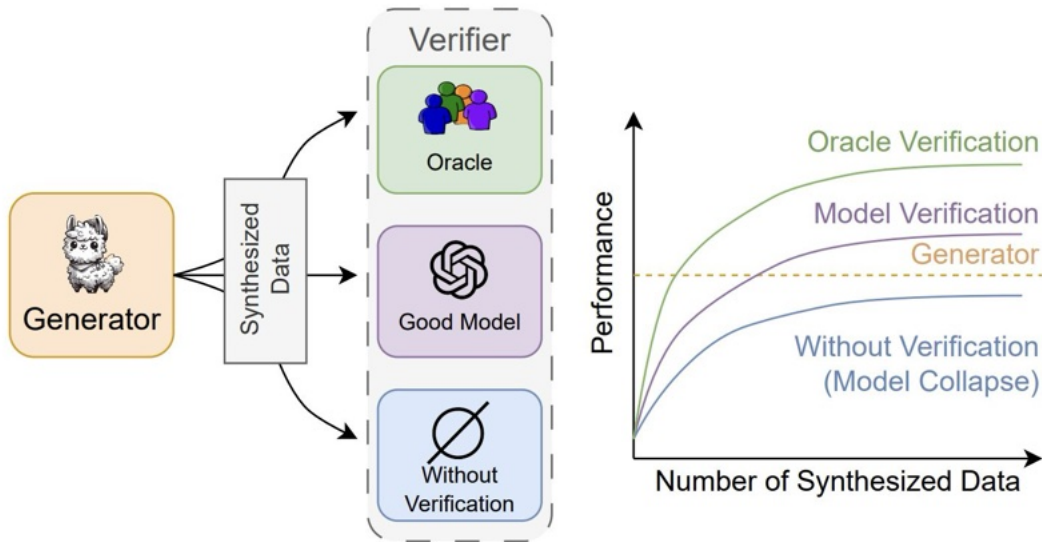
Training: what does it learn?



=> More diversity in multi-turn, no need for CoT prompting

Execution feedback for grounding

- EF is a playground for “model verification” research
- Enables synthetic data and RLEF training at scale!



Feng, Dohmatob, Yang, Charton, Kempe (2024)

Code World Model: Building World Models for Computation

Gabriel Synnaeve
Research @ FAIR
Meta Superintelligence Labs (MSL)

The Team

The FAIR Codegen Team at Meta

Core contributors

Jade Copet
Quentin Carbonneaux
Gal Cohen
Jonas Gehring

Jacob Kahn
Jannik Kossen
Felix Kreuk
Emily McMilin

Michel Meyer
Yuxiang Wei
David Zhang
Kunhao Zheng

Contributors

Jordi Armengol-Estapé
Pedram Bashiri
Maximilian Beck
Pierre Chambon
Abhishek Charnalia
Chris Cummins
Juliette Decugis
Zacharias V. Fisches
François Fleuret
Fabian Gloeckle
Alex Gu
Michael Hassid

Daniel Haziza
Badr Youbi Idrissi
Christian Keller
Rahul Kindi
Hugh Leather
Gallil Maimon
Aram Markosyan
Francisco Massa
Pierre-Emmanuel Mazaré
Vegard Mella
Naila Murray
Keyur Muzumdar

Peter O'Hearn
Matteo Pagliardini
Dmitrii Pedchenko
Tal Remez
Volker Seeker
Marco Selvi
Oren Sultan
Sida Wang
Luca Wehrstedt
Ori Yoran
Lingming Zhang

Senior core contributors

Taco Cohen

Yossi Adi

Gabriel Synnaeve

The Goal: Models that Reason, Plan, and Make Decisions

- Predict future ***observations*** given past ***observations*** and ***actions***
- Why?
 - **Representation learning**: predicting the future helps to understand the world.
 - **Planning and reasoning**: consider different actions and see if we like the results.
- There's a false dichotomy between “LLMs” and “World Models” – let's use LLMs for WMs!

What does it mean to model code?

Isn't code just syntax?

Syntax

```
def count(s, t):  
    n = 0  
    for c in s:  
        n += int(c == t)  
    return n
```

```
count("strawberry", "r")
```

Execution

This code loops $\text{len}(s)$ times, incrementing the counter n if each time a character in s matches the input character t .

In the first loop iteration:

$n = 0$, $c = 's'$, and $t = 'r'$

...

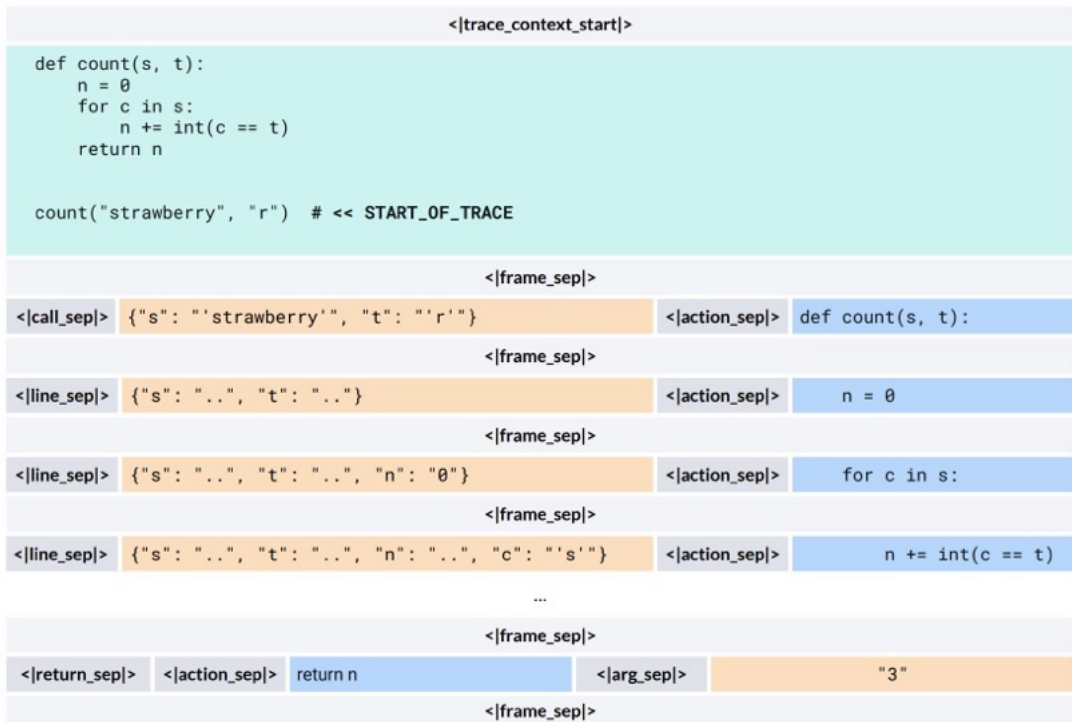
What does it mean to model code?

We want to predict program ***execution!***

Let's implicitly predict a transition function of program states!

What does it mean to model code?

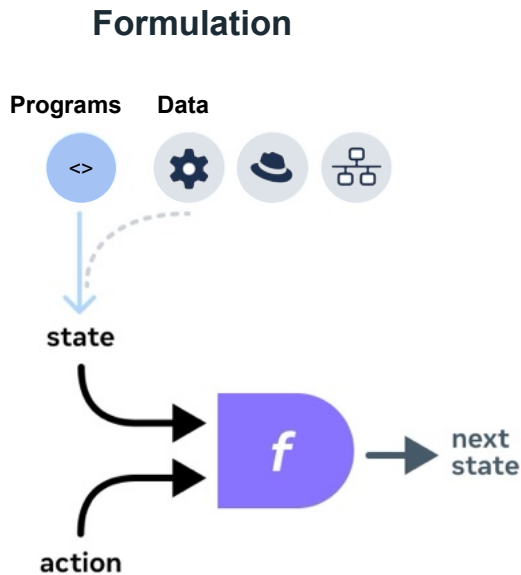
Execution Tracing in Action



Why stop at functions?

- **Trace at the:**
 - Function-level
 - Repo-level
 - Distributed-system-level
 - CodeContest solutions
 - Natural-language tracing
 - ...

Modeling a Program Transition Function

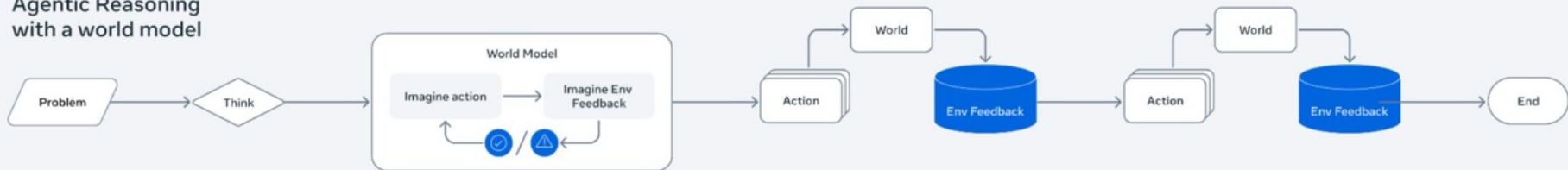


The Broader Approach: World Modeling

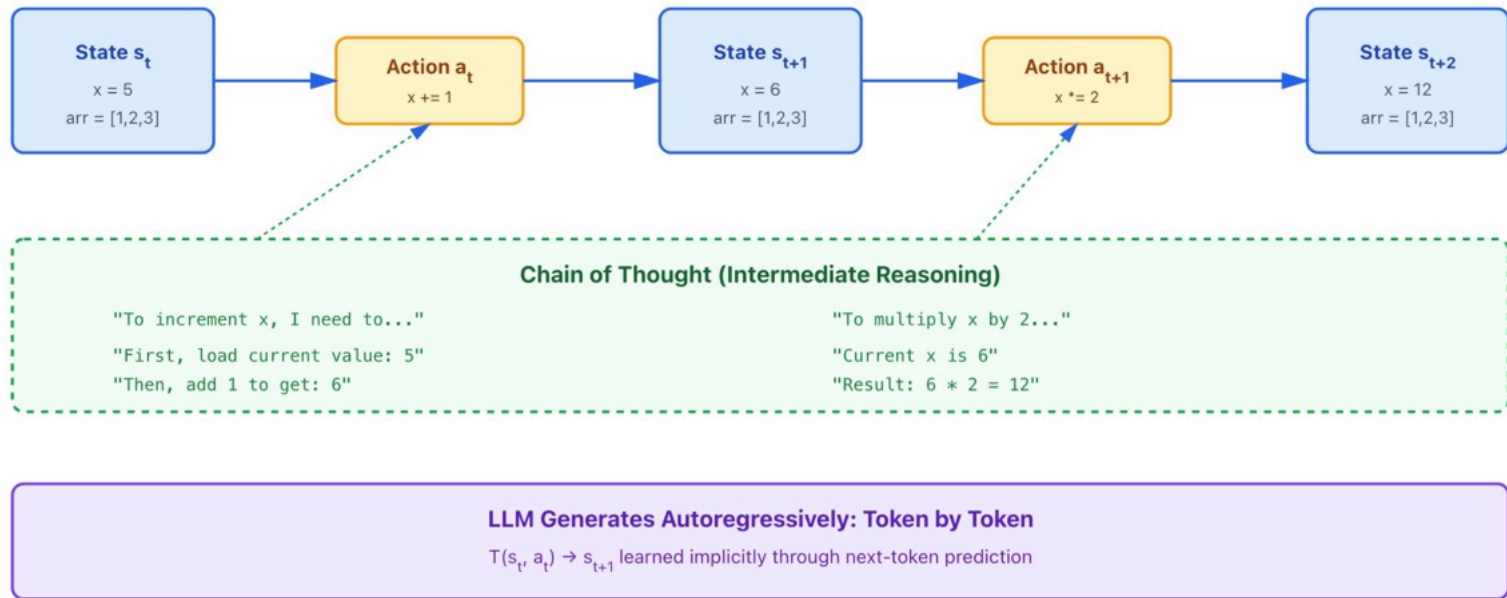
Agentic Reasoning



Agentic Reasoning with a world model



World Modeling with an Autoregressive LLM



Execution traces provide (state, action, next_state) tuples for training

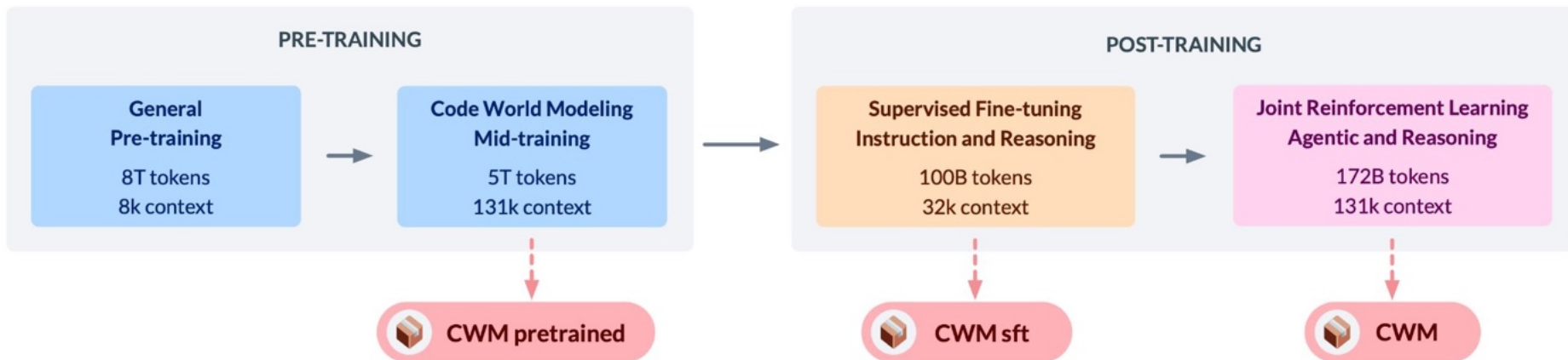
Data: Executable Repo Images



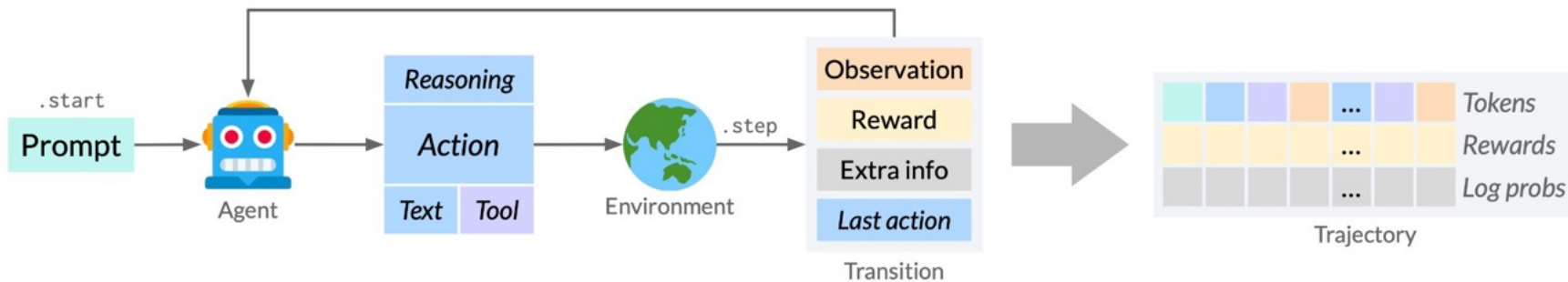
The Artifact: Code World Model (CWM)

- **32B parameter dense** transformer for research
- **131,072** context length for long reasoning sequences
- **Trained end-to-end**: data, pre-training, mid-training, post-training (RL)

Training CWM: Process



Parameterizing the Problem: Agentic RL



The diagram illustrates the workflow of the Agent in the PatchBench framework. The process starts with an **Issue** (represented by a green exclamation mark icon) which is input to the **Agent** (represented by a blue robot icon). The Agent interacts with the **Repo sandbox** (represented by a terminal icon) to execute bash commands. The Agent's actions are categorized into **Custom plugin tools** (represented by a red box) and **Trajectory** (represented by a blue box).

Custom plugin tools include:

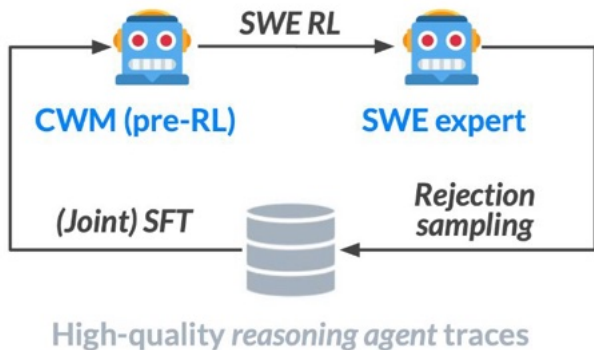
- <tool: edit>**: Replaces `[search_lines]` from `[path]` with `[replacement_lines]`.
- <tool: create>**: Creates a new file at `[path]` with `[content]`.
- <tool: submit>**: Marks `[result]` as your final submission.

Trajectory includes:

- Reasoning
- <tool: ...>
- Feedback
- ...
- <tool: submit>
- Feedback
- Summary

The Agent's output is a **Predicted patch** (represented by a blue plus icon), which is compared against the **Test suite** (represented by a black greater-than icon) and the **Oracle patch** (represented by a green plus icon) to calculate the **Reward: tests or patch similarity (if tests fail)**.

SWE-RL Bootstrapping



<think> The instantiation logic is in ... The key function here is ... But wait, where is `\_partial\_` popped from the config?... Let me outline the steps: ... Hold on, in the current code, `\_recursive\_` and `\_convert\_` are popped ... Looking at the code ...

```
if OmegaConf.is_dict(node):
    convert = node[_Keys.CONVERT] ...
```

Hang on, but in the code flow, `\_call\_target` is called when the node has a `\_target\_` ... So the steps are: ... **</think>**

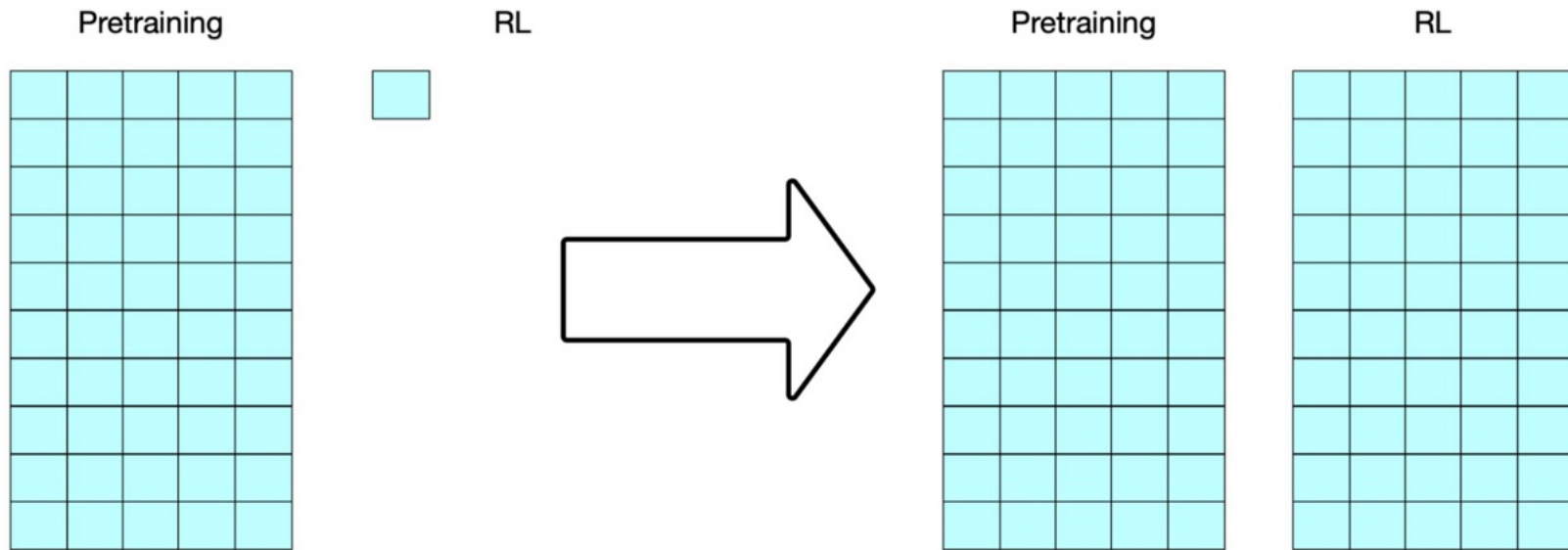
Reasoning

<tool: bash>

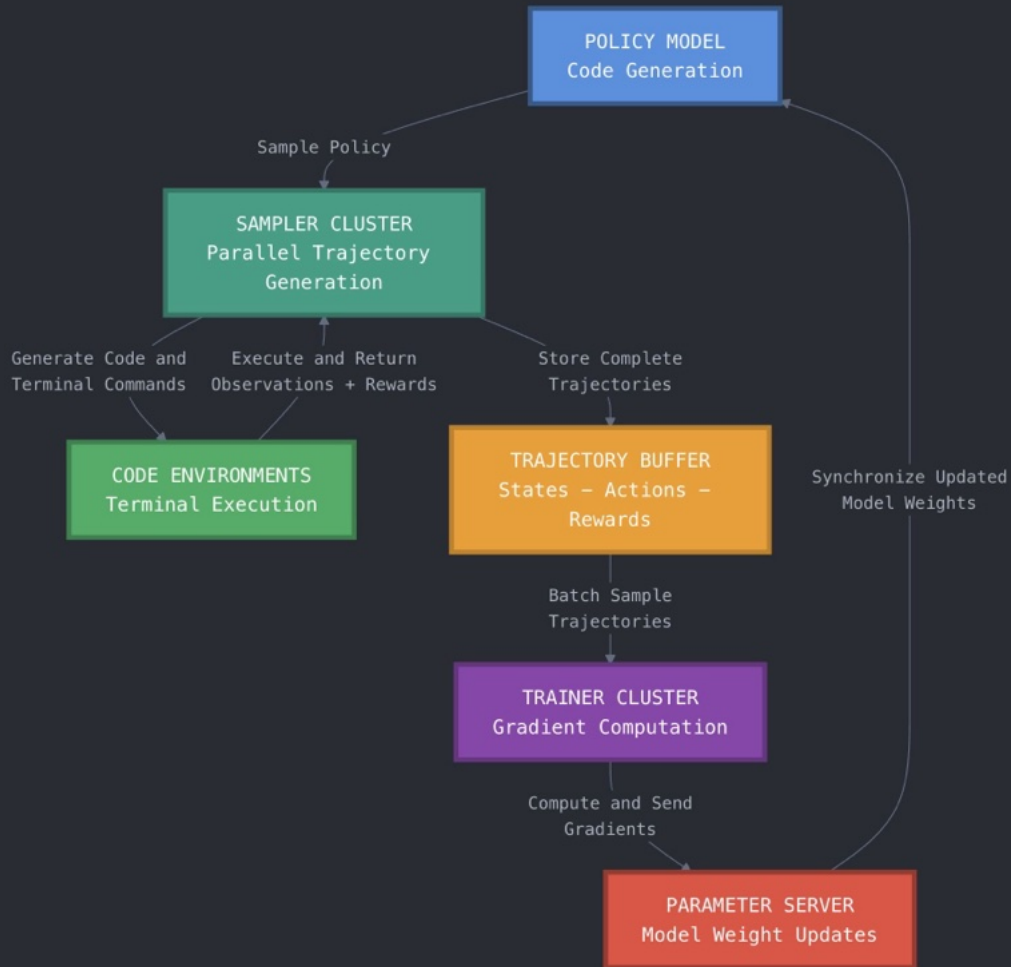
```
grep -n "_Keys(" hydra/_internal/instantiate/_instantiate2.py
</tool>
```

Tool call

Our New Goal in RL: Scale



The Distributed RL System at Scale

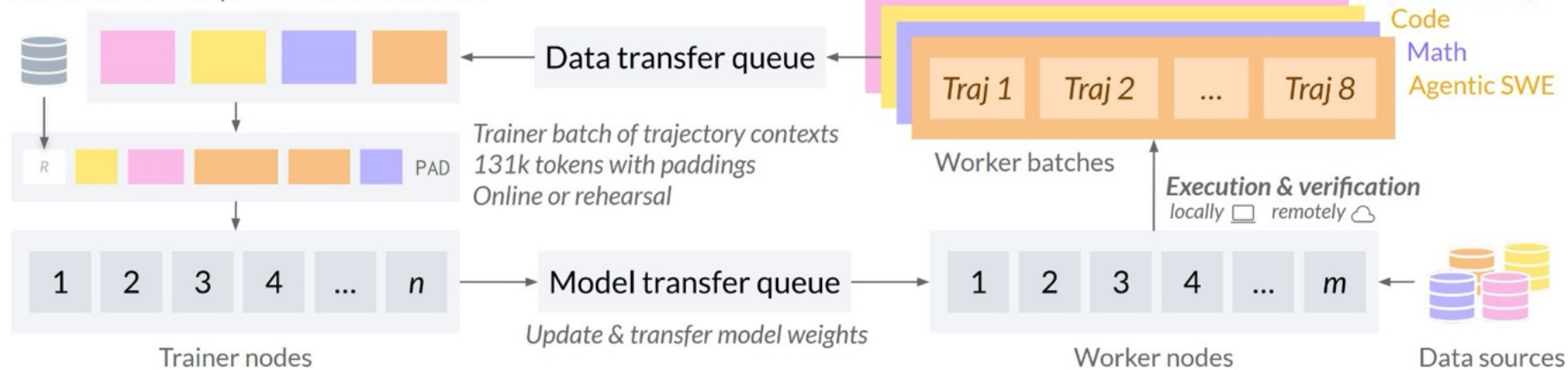


A Challenge in Scaling Async RL

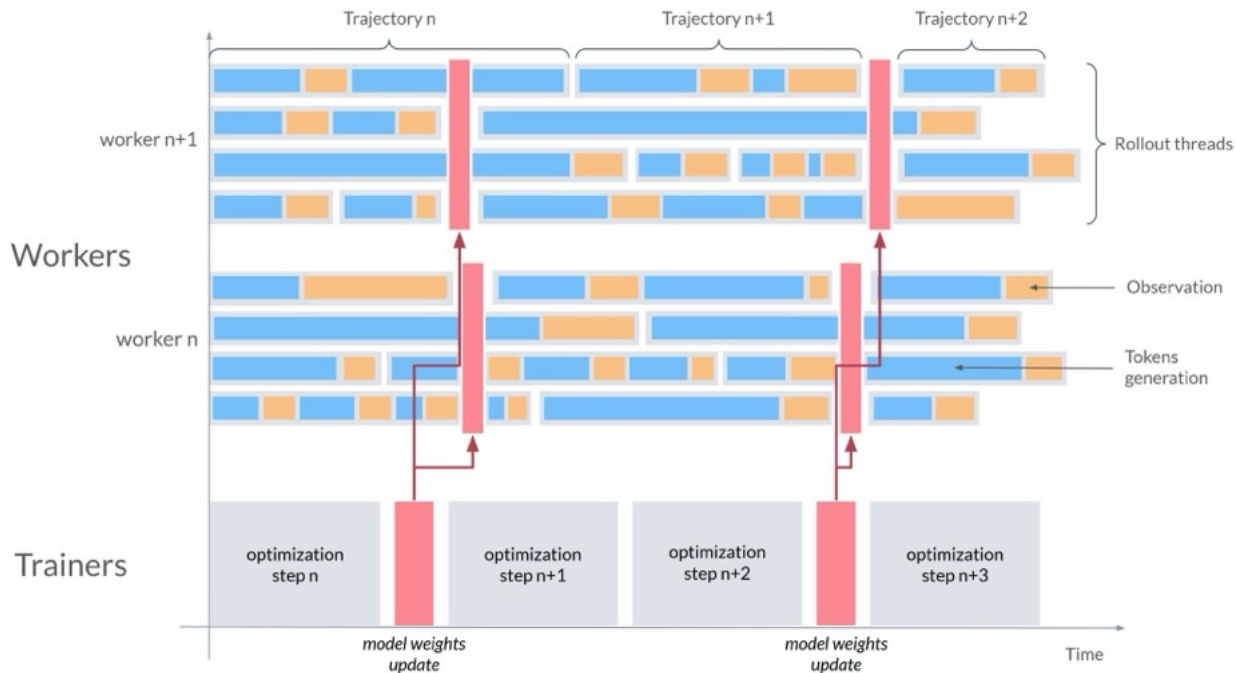
- This is a **produce-consume** pipeline problem!
 - ◆ Samplers **produce** trajectories by running expensive environment interactions
 - ◆ Trainers **consume** trajectories to compute gradients
 - ◆ Model weights must be **synchronized** across distributed workers

Async RL: Design

Rehearsal & Local queue of worker batches



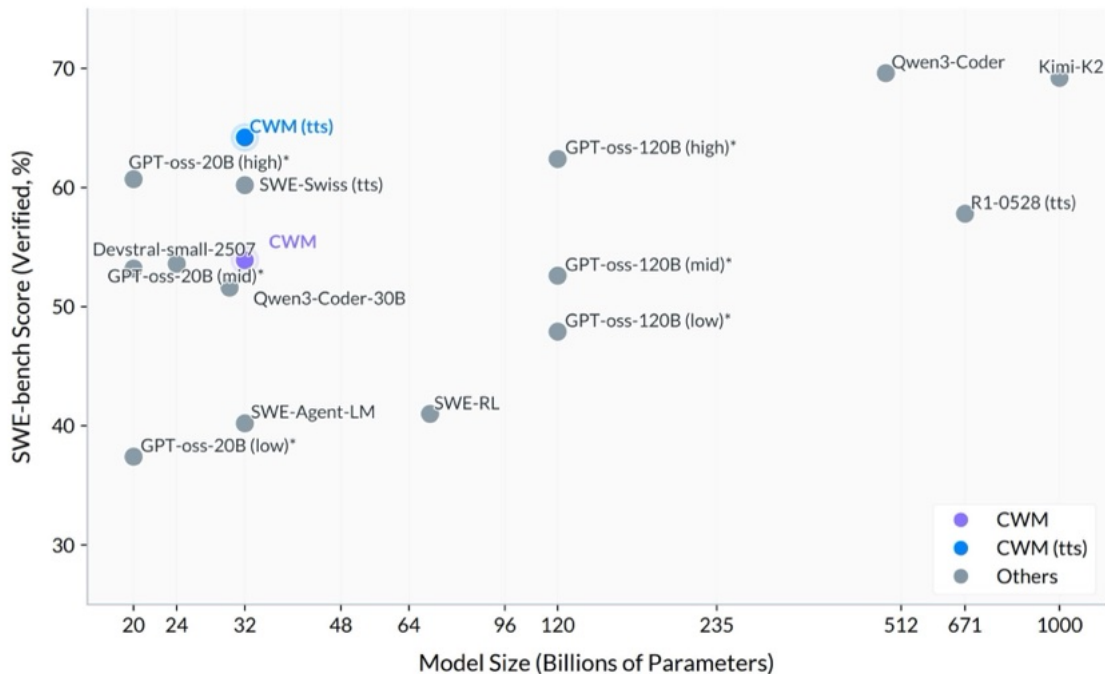
Async RL: Updating Mid-Trajectory

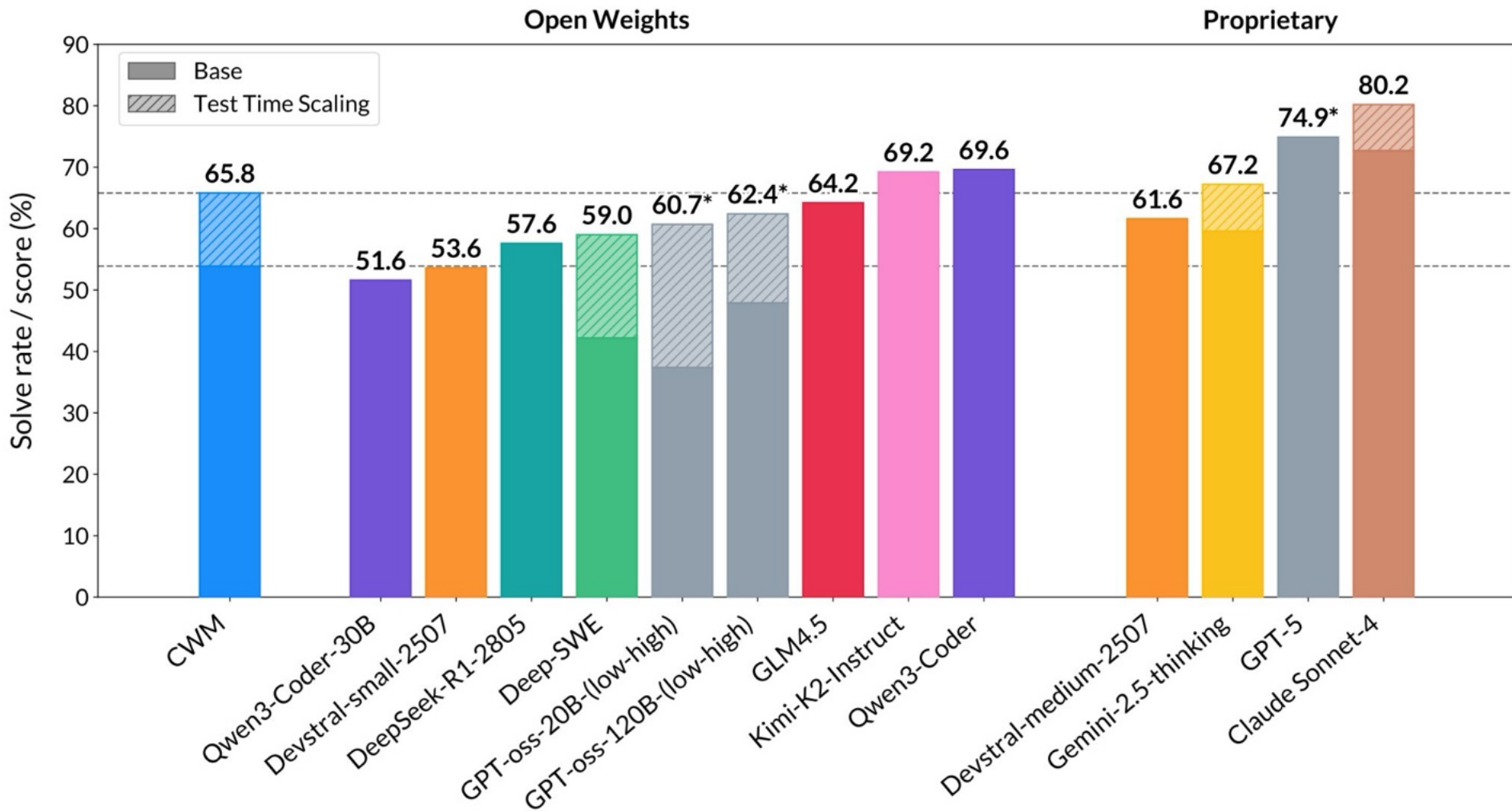


CWM's RL Post-Training Adventure

- $O(20k)$ steps (batches)
- $O(2k-4k)$ GPUs (HGX, H100, IB)
- $O(200B)$ tokens processed

Voilà – CWM (Copet et al. 2025) is a strong model!





*What can we **do** with a model that
understands program execution traces?*

What can we do with CWM?

CWM Traces Code [Well]

```
<|begin_of_text|><|trace_context_start|>
def f(nums):
    a = 0
    for i in range(len(nums)):
        nums.insert(i, nums[a])
        a += 1
    return nums

def main(): # << START_OF_TRACE
    return f([1, 3, -1, 1, -2, 6])

<|frame_sep|><|call_sep|>{}<|action_sep|>def main():

<|frame_sep|><|line_sep|>{}<|action_sep|>return f([1, 3, -1, 1, -2, 6])

<|frame_sep|><|call_sep|>{"nums": "[1, 3, -1, 1, -2, 6]"}<|action_sep|>def f(nums):

<|frame_sep|><|line_sep|>{"nums": "[1, 3, -1, 1, -2, 6]"}<|action_sep|>a = 0

<|frame_sep|><|line_sep|>{"nums": "[1, 3, -1, 1, -2, 6]", "a": "0"}<|action_sep|>for i in range(len(nums)):

<|frame_sep|><|line_sep|>{"nums": "[1, 3, -1, 1, -2, 6]", "a": "0", "i": "0"}<|action_sep|>nums.insert(i, nums[a])

<|frame_sep|><|line_sep|>{"nums": "[1, 1, 3, -1, 1, -2, 6]", "a": "0", "i": "0"}<|action_sep|>a += 1

<|frame_sep|><|line_sep|>{"nums": "[1, 1, 3, -1, 1, -2, 6]", "a": "1", "i": "0"}<|action_sep|>for i in range(len(nums)):

[... iterations continue with nums growing to [1, 1, 1, 1, 1, 1, 1, 3, -1, 1, -2, 6] ...]

<|frame_sep|><|return_sep|><|action_sep|>return nums<|arg_sep|>"[1, 1, 1, 1, 1, 1, 1, 3, -1, 1, -2, 6]"

<|frame_sep|><|return_sep|><|action_sep|>return f([1, 3, -1, 1, -2, 6])<|arg_sep|>"[1, 1, 1, 1, 1, 1, 1, 3, -1, 1, -2, 6]"
```

Neural Debugging

Traditional Debugger (pdb) ✗ Failed

```
1 def merge_sort (arr):
2     if len (arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     # TODO: implement merge logic
6     left = ???
7     right = ???
8     return merge(left, right)
9
```

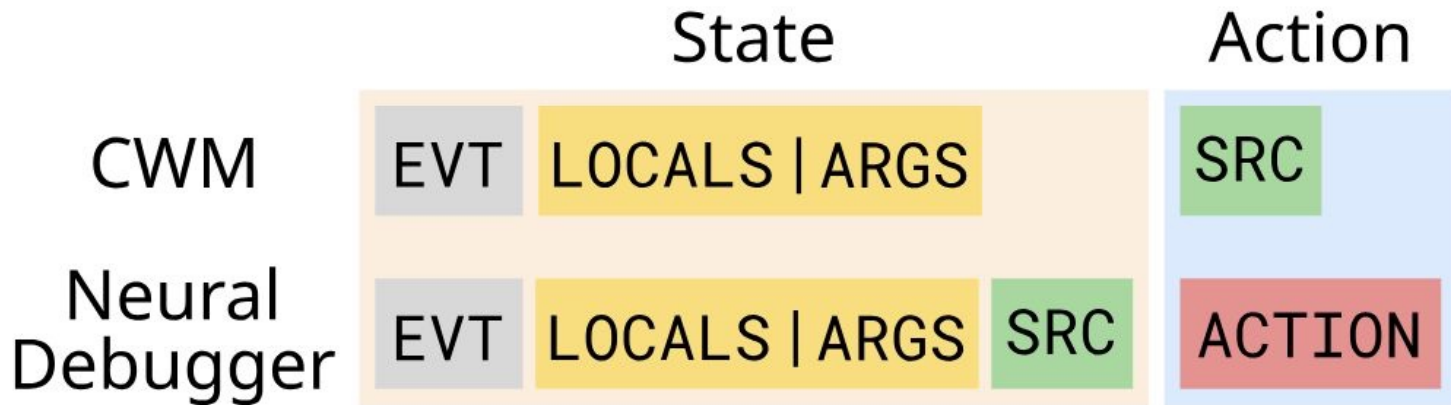
```
(Pdb) n
> /tmp/code.py (7)merge_sort ()
-> left = ???
(Pdb) p left
*** NameError: name 'left' is not defined
(Pdb) s
*** NameError: Cannot step into undefined code
Can't trace incomplete code!
Missing implementation blocks execution...
```

CWM Neural Tracer ✓ Success

```
1 def merge_sort (arr):
2     if len (arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     # TODO: implement merge logic
6     left = ???
7     right = ???
8     return merge(left, right)
9
```

```
>>> cwm.trace ("merge_sort" , arr=[3,1,4])
Simulating execution trace...
Step 1: arr=[ 3,1,4], mid= 1
Step 2: left = merge_sort([ 3]) = [3]
Step 3: right = merge_sort([ 1,4]) = [1,4]
Step 4: merge([ 3], [1,4]) = [1,3,4]
✓ Predicted execution:[ 3,1,4] → [ 1,3,4]
Inferred from 200M+ traced functions
```


How to replace PDB?



Forward and inverse execution

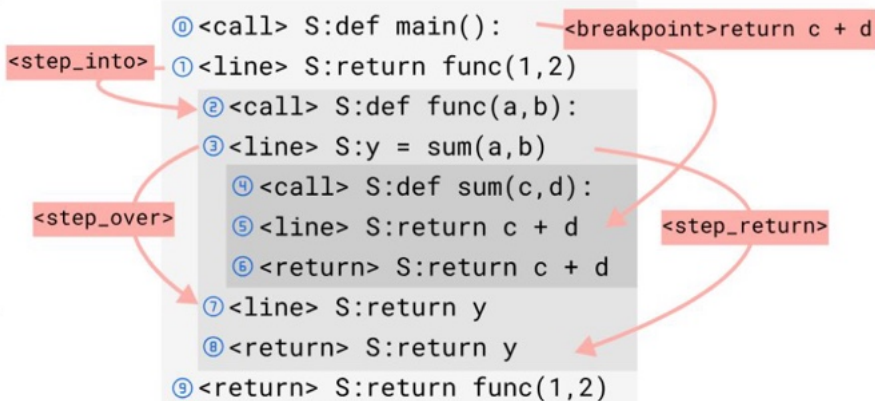
code.py

```
def sum(c,d):
    return c + d

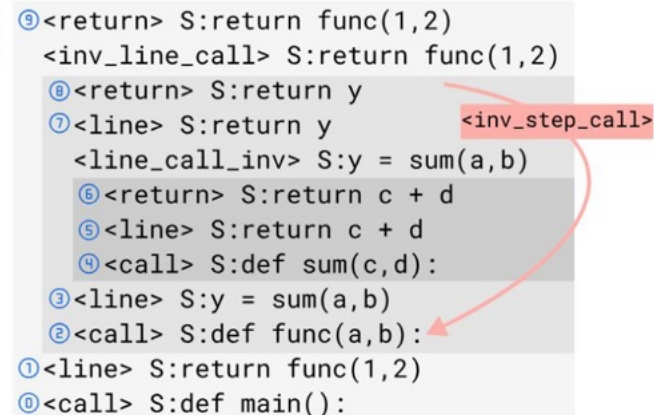
def func(a,b):
    y = sum(a,b)
    return y

def main():
    return func(1,2)
```

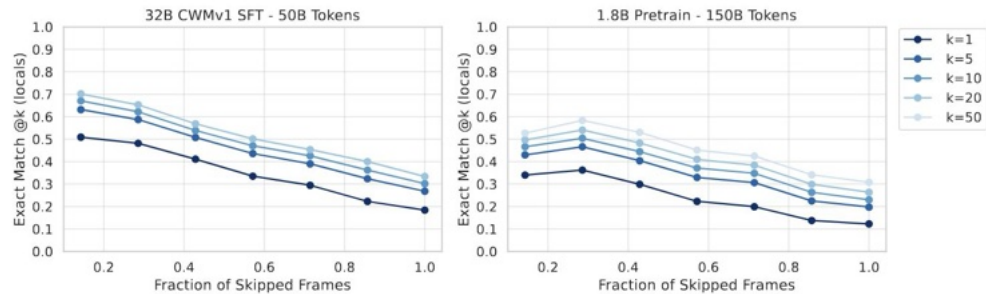
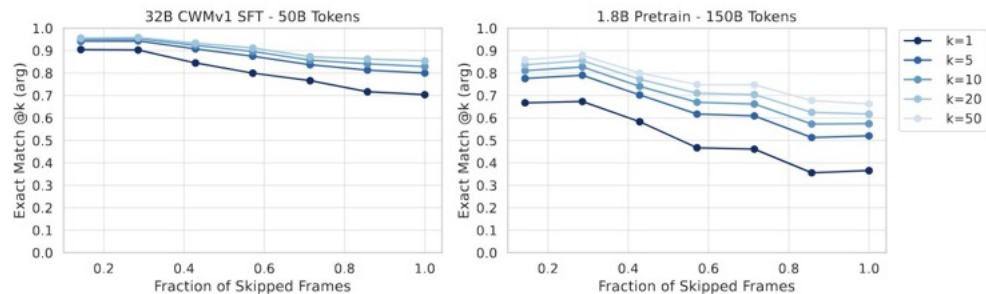
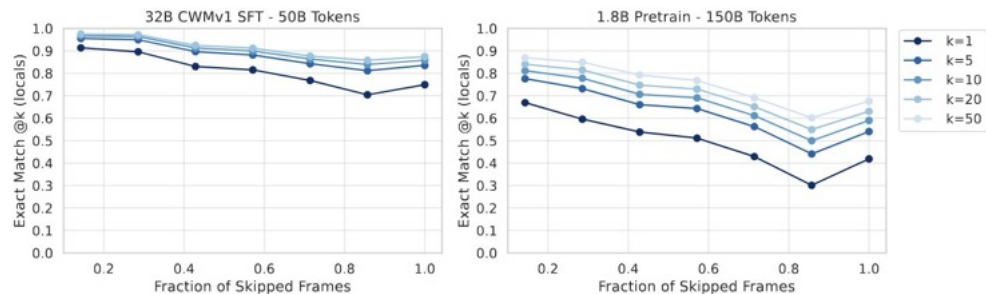
Forward State Tree:



Inverse State Tree:



Forward execution (predicting future states and outputs), and
inverse execution (inferring prior states or inputs), conditioned on debugger actions

(a) Input prediction with inverse `inv_step_call` action (exact match of local variable dict).(b) Output prediction with forward `step_return` action (exact match of the return argument).(c) Output prediction with forward `breakpoint` action (exact match of local variable dict).

The Halting Problem

*Given a program and an input,
will a program run forever?*

```
CWM Neural Approximation ≈ Probabilistic

1 def collatz (n):
2     while n != 1:
3         if n % 2 == 0:
4             n = n // 2
5         else :
6             n = 3* n + 1
7     return True
8
9 # Does this halt for all n?

>>> cwm.predict_halt (collatz, 27)

Simulating execution trace...

Trace: n= 27 → 82 → 41 → 124 → ...

Pattern: Convergent oscillation detected

✓ Prediction: HALTS (confidence: 0.94)
```

<https://github.com/facebookresearch/halteval>

Go do things!

We built CWM for research and building! *Go forth!*

[huggingface://facebook/cwm](https://huggingface.co/facebook/cwm)



[github://facebookresearch/cwm](https://github.com/facebookresearch/cwm)



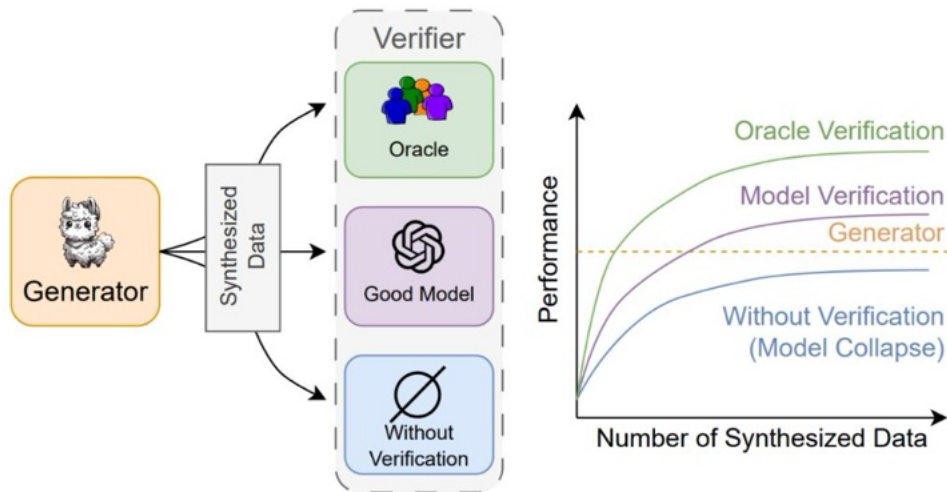
[Technical Report](#)



Bonus slides on LLMs

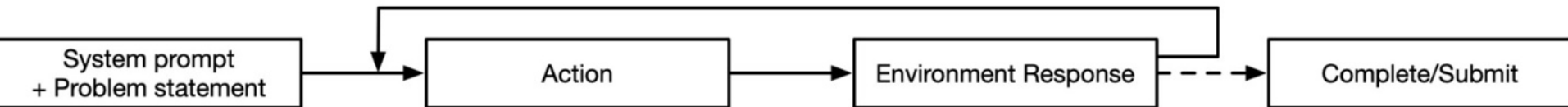
LLMs work so well, but...

- Limits of data scaling
=> synthetic data (incl. RLEF)



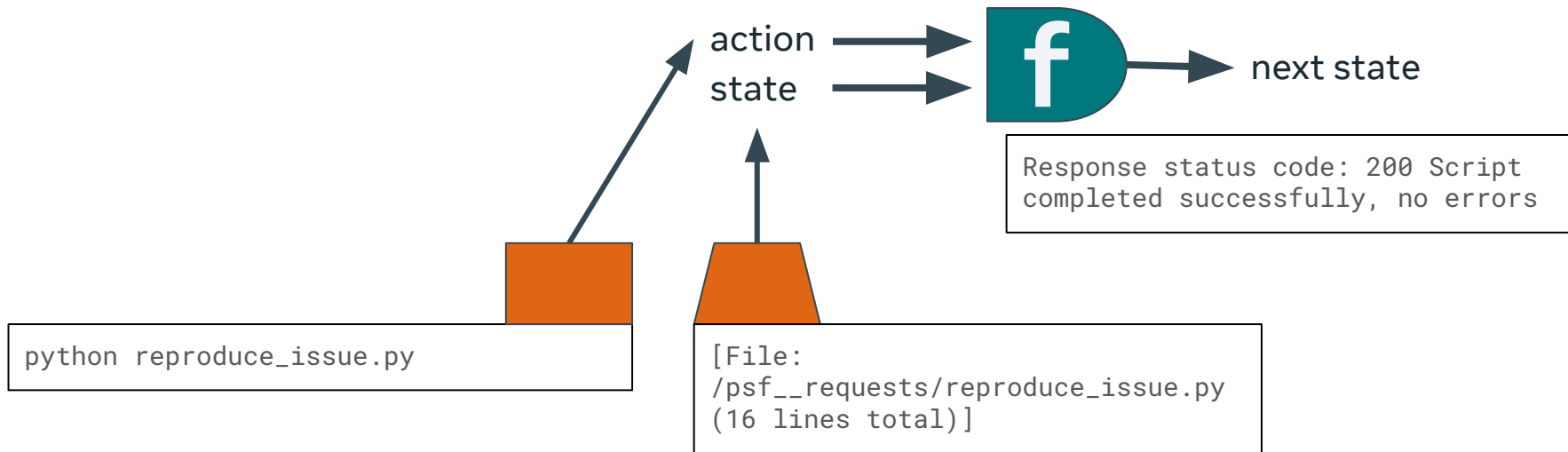
Feng, Dohmatob, Yang, Charton, Kempe (2024)

- Agentic uses of models:
Take actions in an environment

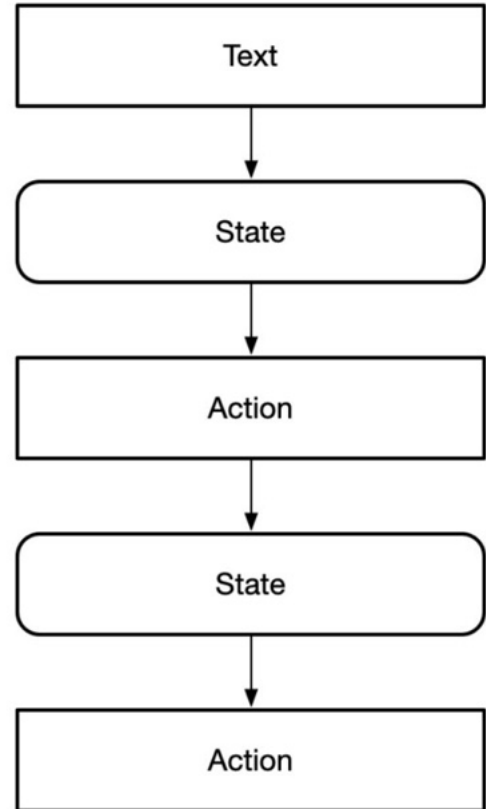
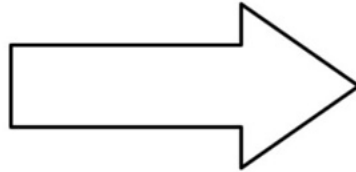
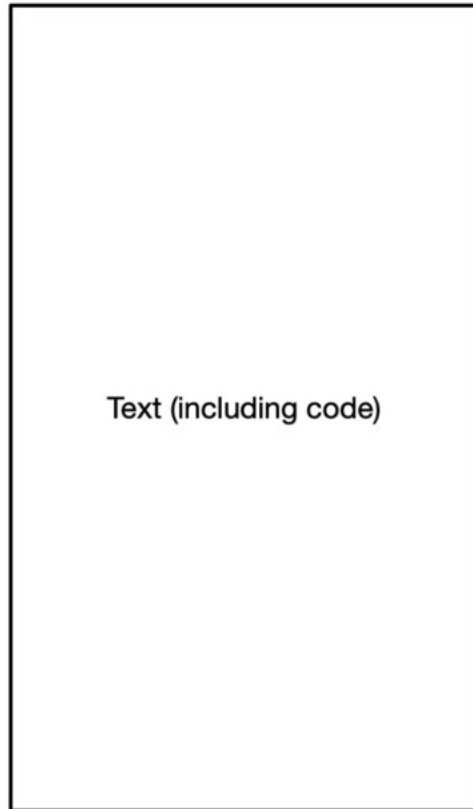


Computational World Models

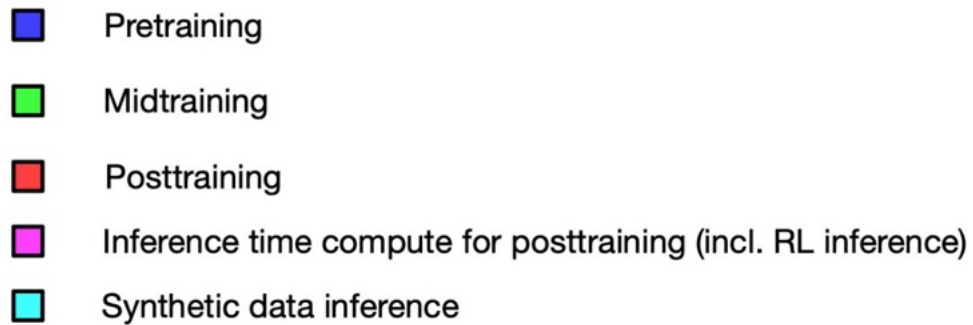
- Model explicitly states and actions
- Learn from execution at pretraining
- More in-domain for agentic uses
- Enable model-based and hierarchical planning



What do we change? Data



Where are training flops spent now (fake numbers)



MoE

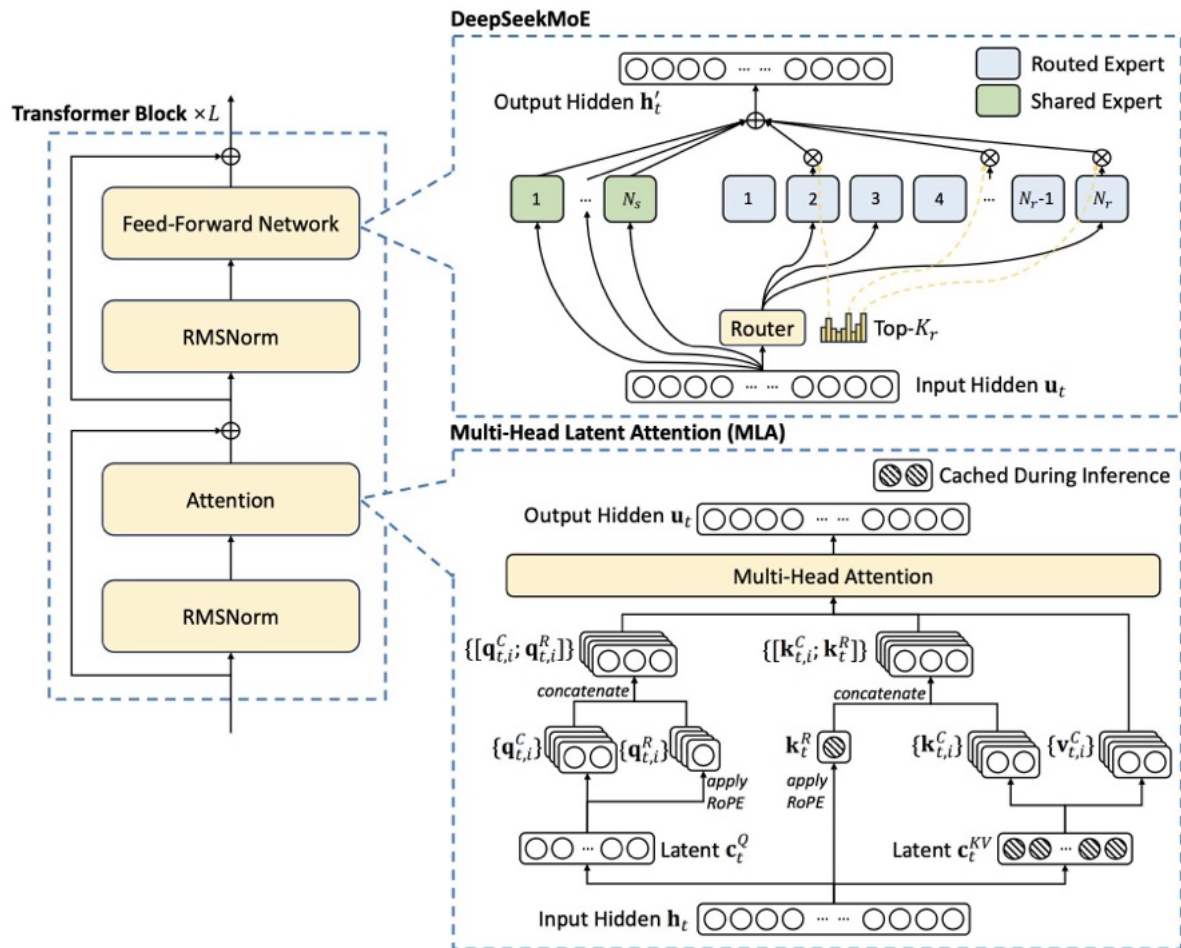


Figure 2 | Illustration of the basic architecture of DeepSeek-V3. Following DeepSeek-V2, we adopt MLA and DeepSeekMoE for efficient inference and economical training.

Parallelism

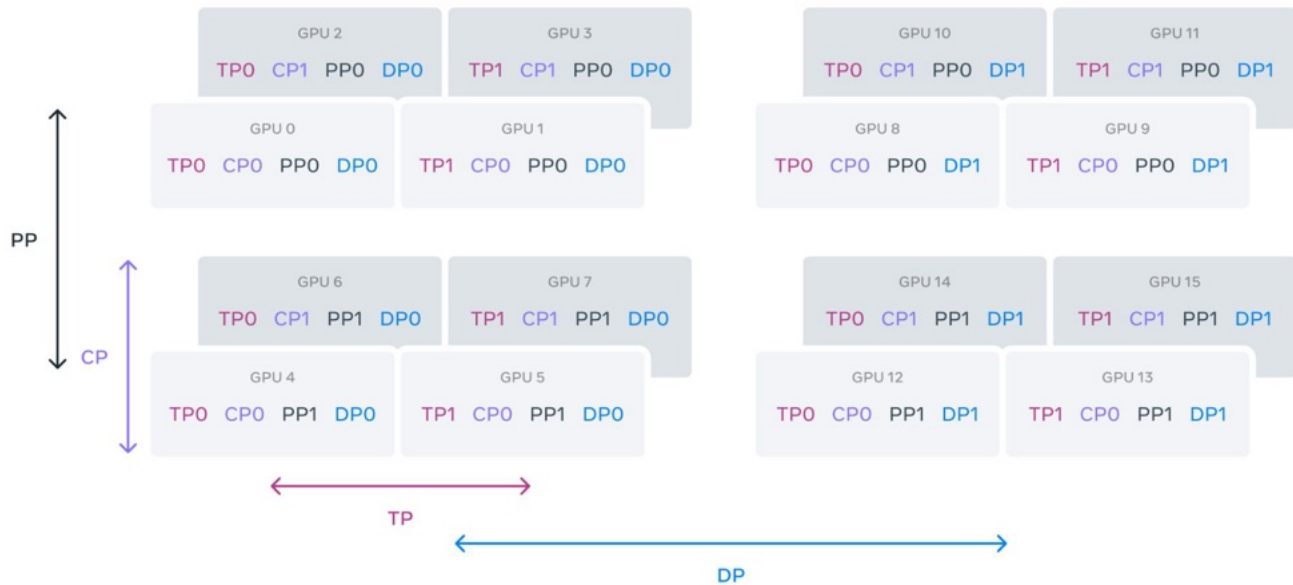


Figure 5 Illustration of 4D parallelism. GPUs are divided into parallelism groups in the order of [TP, CP, PP, DP], where DP stands for FSDP. In this example, 16 GPUs are configured with a group size of $|TP|=2$, $|CP|=2$, $|PP|=2$, and $|DP|=2$. A GPU's position in 4D parallelism is represented as a vector, $[D_1, D_2, D_3, D_4]$, where D_i is the index on the i -th parallelism dimension. In this example, GPU0[TP0, CP0, PP0, DP0] and GPU1[TP1, CP0, PP0, DP0] are in the same TP group, GPU0 and GPU2 are in the same CP group, GPU0 and GPU4 are in the same PP group, and GPU0 and GPU8 are in the same DP group.

Comms

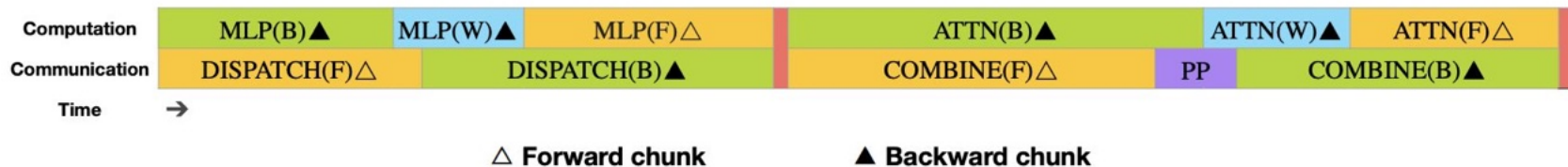


Figure 4 | Overlapping strategy for a pair of individual forward and backward chunks (the boundaries of the transformer blocks are not aligned). Orange denotes forward, green denotes "backward for input", blue denotes "backward for weights", purple denotes PP communication, and red denotes barriers. Both all-to-all and PP communication can be fully hidden.

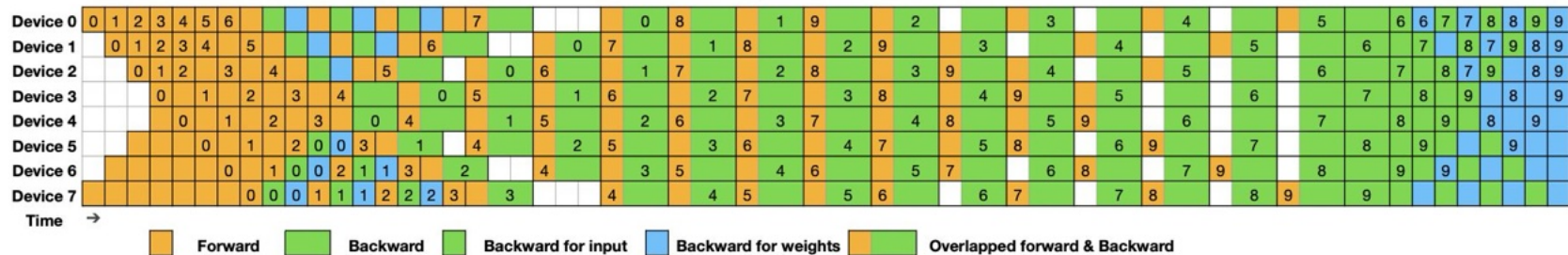


Figure 5 | Example DualPipe scheduling for 8 PP ranks and 20 micro-batches in two directions. The micro-batches in the reverse direction are symmetric to those in the forward direction, so we omit their batch ID for illustration simplicity. Two cells enclosed by a shared black border have mutually overlapped computation and communication.

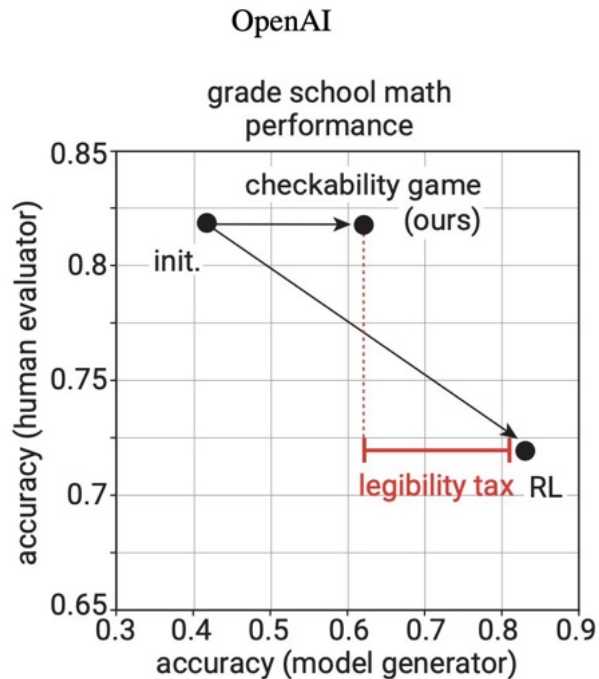
Method	Bubble	Parameter	Activation
1F1B	$(PP - 1)(F + B)$	$1\times$	PP
ZB1P	$(PP - 1)(F + B - 2W)$	$1\times$	PP
DualPipe (Ours)	$(\frac{PP}{2} - 1)(F\&B + B - 3W)$	$2\times$	$PP + 1$

Table 2 | Comparison of pipeline bubbles and memory usage across different pipeline parallel methods. F denotes the execution time of a forward chunk, B denotes the execution time of a full backward chunk, W denotes the execution time of a "backward for weights" chunk, and $F\&B$ denotes the execution time of two mutually overlapped forward and backward chunks.

PROVER-VERIFIER GAMES IMPROVE LEGIBILITY OF LLM OUTPUTS

Jan Hendrik Kirchner* Yining Chen*

Harri Edwards[†] Jan Leike[†] Nat McAleese Yuri Burda[†]



init



Shawna's father is five times as old as Shawna. Shawna is currently three times as old as Aliya. If Aliya is 3 years old, how old is Shawna's father?



It follows that Shawna is currently $3 \times 2 = 6$ years old.

Subsequently this implies that Shawna's father is currently $6 \times 5 = 30$ years old.

Answer: 30

round 1 of checkability training



Shawna's father is five times as old as Shawna. Shawna is currently three times as old as Aliya. If Aliya is 3 years old, how old is Shawna's father?



First we need to determine the age of Aliya. From the problem description, it says that Aliya is 3 years old.

Then we need to determine the age of Shawna. From the problem description, we know that Shawna is currently three times as old as Aliya, so the age of Shawna is 3 times 3, or $3 \times 3 = 9$.

Next, we need to determine Shawna's father. The problem states that Shawna's father is five times as old as Shawna. Therefore, the age of Shawna's father becomes 5 times 9, or $5 \times 9 = 45$.

Answer: 45

round 5 of checkability training



Shawna's father is five times as old as Shawna. Shawna is currently three times as old as Aliya. If Aliya is 3 years old, how old is Shawna's father?



We start by calculating the age of Aliya.

It says that Aliya is 3 years old.

Then, we need to calculate Shawna's current age.

Shawna is currently three times as old as Aliya. Thus, Shawna's current age is $3 \times 3 = 9$.

Now we need to calculate Shawna's father's age.

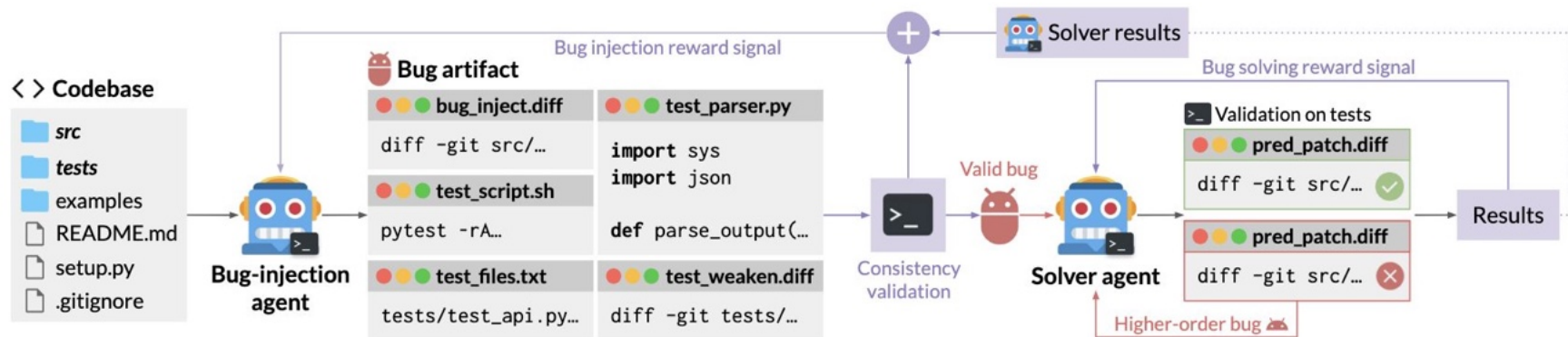
Shawna's father is five times as old as Shawna. Therefore, Shawna's father's age is $5 \times 9 = 45$.

Therefore Shawna's father's age is 45.

Answer: 45

Toward Training Superintelligent Software Agents through Self-Play SWE-RL

Yuxiang Wei^{13†*} Zhiqing Sun^{2†} Emily McMilin^{1†} Jonas Gehring¹ David Zhang¹
Gabriel Synnaeve¹ Daniel Fried^{14†} Lingming Zhang^{3†} Sida Wang^{1†}



Examples

test_weakening_patch.diff

```
@@ -957,6 +957,5 @@ class TestProphetWarmStart:
     daily_univariate_ts.iloc[:500], show_progress=False
 )
     m2 = Prophet(mcmc_samples=100, stan_backend=backend).fit(
-         daily_univariate_ts.iloc[:510], init=warm_start_params(m), show_progress=False
+         daily_univariate_ts.iloc[:510], show_progress=False
     )
-     assert m2.params["delta"].shape == (200, 25)
```

The `Column` class has various methods for creating filters... **Removing a key method or property, such as the `\_\_eq\_\_` method, could break tests** that compare columns, like `test\_column` in `test\_types.py`...

<tool:edit> {remove code hunks from the files...}

```
- class SlurmParseException(Exception):                                Removal-oriented bug
-     pass

- def _expand_id_suffix(suffix_parts: str) -> List[str]:
-     """Parse the a suffix formatted like..."""
-     suffixes = []
-     for suffix_part in suffix_parts.split(","):
-         if "-" in suffix_part:
-             low, high = suffix_part.split("-")
-             for num in range(int(low), int(high) + 1):
-                 suffixes.append(str(num))
-         else:
-             suffixes.append(suffix_part)
-     return suffixes
```

oracle_test_patch.diff (weakening patch reversed: git diff buggy original -- {test_files})

```
@@ -957,5 +957,6 @@ class TestProphetWarmStart:
     daily_univariate_ts.iloc[:500], show_progress=False
 )
     m2 = Prophet(mcmc_samples=100, stan_backend=backend).fit(
-         daily_univariate_ts.iloc[:510], show_progress=False
+         daily_univariate_ts.iloc[:510], init=warm_start_params(m), show_progress=False
     )
+     assert m2.params["delta"].shape == (200, 25)
```

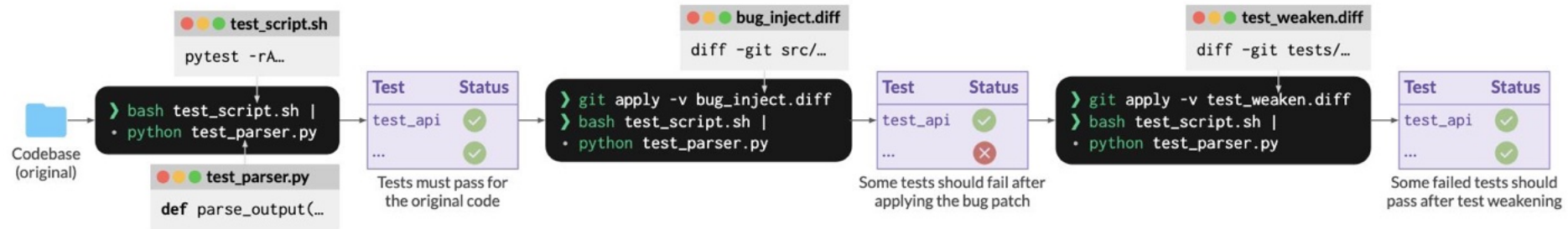
Let me investigate the "Speed up `fourier\_series` (#2334)" commit, as performance optimizations can sometimes **introduce bugs or dependencies that might be interesting to revert**. I'll use `git show` to view the changes...

<tool:bash> git show e2b4ef3

<tool:edit> {revert history changes...}

```
def fit(self, stan_init, stan_data, **kwargs):                                History-aware bug
    if 'inits' not in kwargs and 'init' in kwargs:
-         stan_init = self.sanitize_custom_inits(stan_init, kwargs['init'])
+         kwargs['inits'], _ = self.prepare_data(kwargs['init'], stan_data)
+         del kwargs['init']
-         inits_list, data_list = self.prepare_data(stan_init, stan_data)
+         stan_init, stan_data = self.prepare_data(stan_init, stan_data)
        args = dict(
-         data=data_list,
-         inits=inits_list,
-         algorithm='Newton' if data_list['T'] < 100 else 'LBFGS',
+         data=stan_data,
+         inits=stan_init,
+         algorithm='Newton' if stan_data['T'] < 100 else 'LBFGS',
```

Consistency validation. Each first-order bug artifact is validated against a set of execution rules to ensure it is meaningful. **Figure 4** shows some key steps. Below, we describe the complete checks:



Reward design. The bug-injection agent receives rewards based on the quality and difficulty of generated bugs, which we measure through consistency validation and solver performance. Let $s \in [0, 1]$ denote the *solve rate*, defined as the fraction of solver attempts that successfully fix the bug entirely. The bug-injection reward r_{inject} is defined as:

$$r_{\text{inject}} = \begin{cases} -1.0 & \text{if consistency validation fails,} \\ -\alpha & \text{if } s = 0 \text{ or } s = 1, \\ 1 - (1 + \alpha)s & \text{if } 0 < s < 1 \text{ (ideal difficulty),} \end{cases} \quad (1)$$

Constructing the buggy codebase. Once a bug artifact passes the consistency validation, the solver agent will start the bug repair process by interacting with the buggy codebase to produce a bug-fixing prediction patch. **Figure 5** illustrates the pipeline for constructing such buggy codebase.

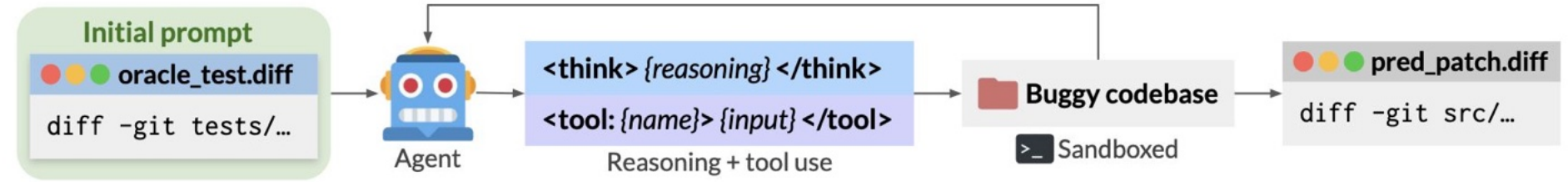
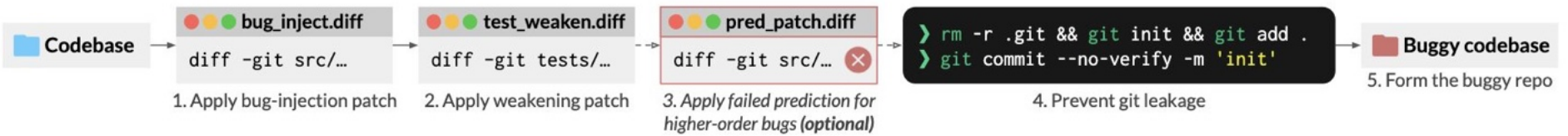


Figure 6: Agentic bug-repair process.

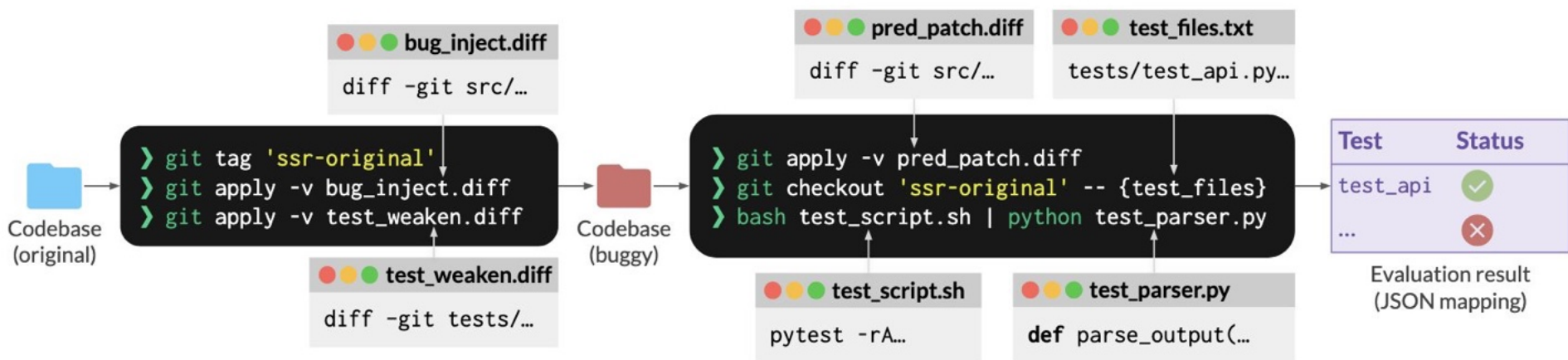
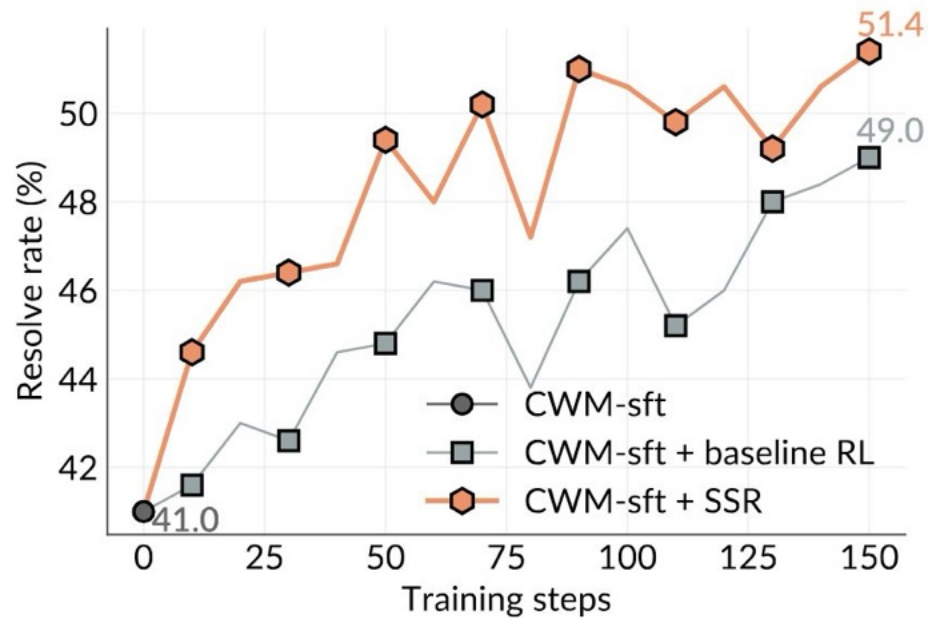
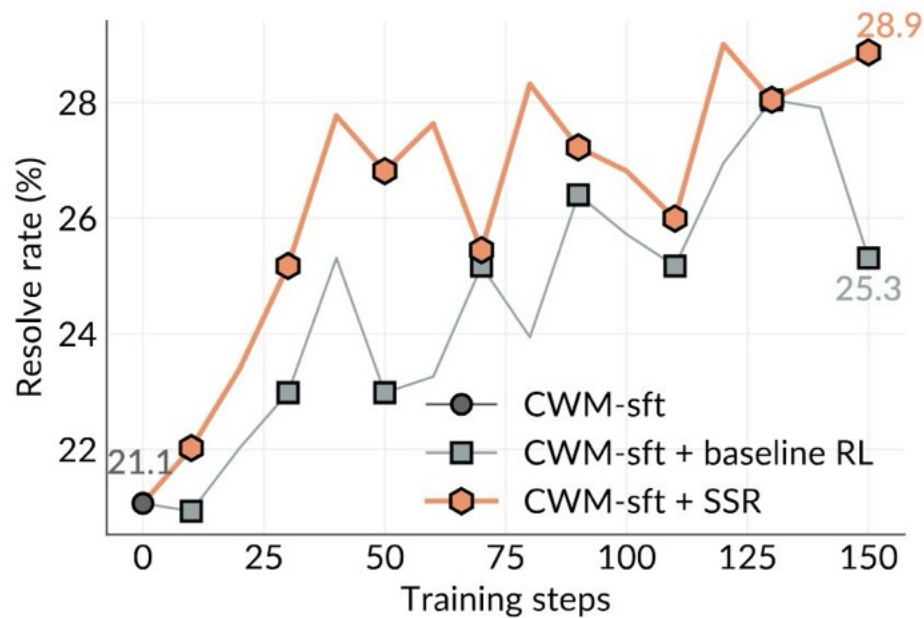


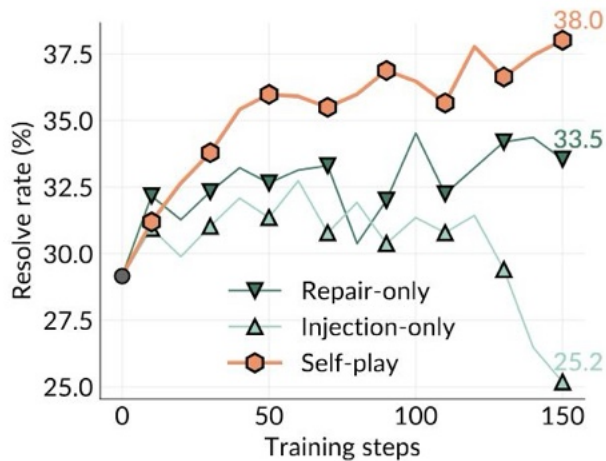
Figure 7: Evaluating the correctness of model-predicted patches.



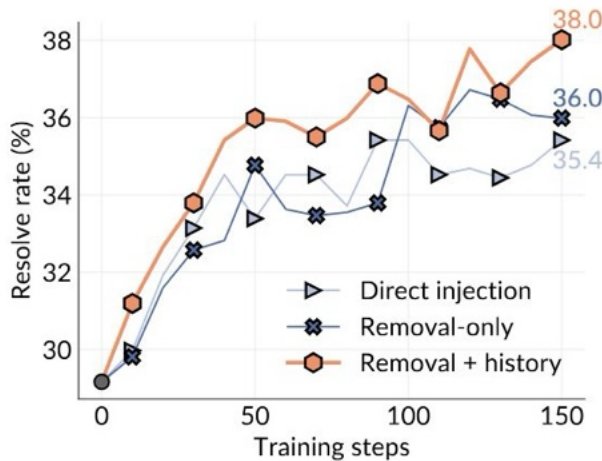
(a) SWE-bench Verified



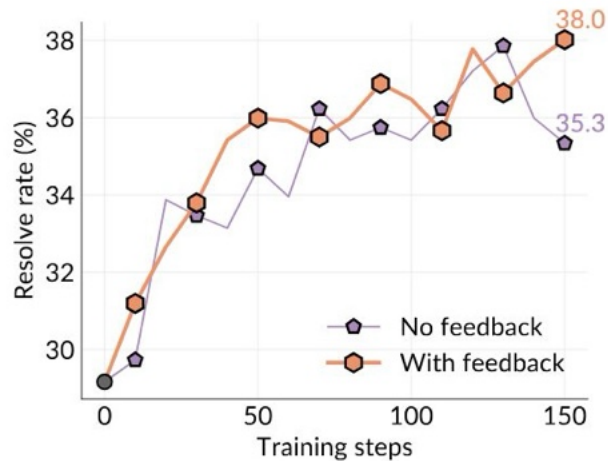
(b) SWE-Bench Pro



(a) Self-play components



(b) Bug-injection methods



(c) Solver feedback in bug-injection

Figure 9: Ablation studies of Self-play SWE-RL. The resolve rate is computed over the union of 500 SWE-bench Verified and 731 SWE-Bench Pro tasks, in total 1231 tasks.